DeepDumps

Premium exam material

Get certification quickly with the **Deep Dumps** Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily.

https://www.DeepDumps.com

About DeepDumps

At DeepDumps, we finally had enough of the overpriced, paywall-ridden exam prep industry. Our team of experienced IT professionals spent years navigating the certification landscape, only to realize that most prep materials came with a price tag bigger than the exam itself. We saw aspiring professionals spending more on study guides than on rent, and frankly, it was ridiculous. So, we decided to change the game. DeepDumps was born with one goal: to provide high-quality exam preparation without draining your wallet.
With DeepDumps, you get expert-vetted study materials, real-world insights, and a fair shot at success—without the financial headache..

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

https://DeepDumps.com

Mail us on - support@deepdumps.com

Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs

Free Exam PDF

You are sure to pass the exam completely free of charge

Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John

★★★★★

Thank you so much for your help. I scored 972 in my exam today. More than 30 questions were there from your PDFs!

Dana

★★★★★

Thanks a lot for this updated AZ-900 Q&A. I just finished my exam and got 374. I followed both of your AZ-900 videos and the 6 PDF, the PDFs are very accurate, all answers are correct. Could you please create a similar video/PDF for DP900, your content-/PDF's is really awesome. The tem'd a really good job. Thank You 😊

Ahamod Shibly

★★★★★

Customer support is realy fast and friendly, I just finished with me exam I passed it along with the 6 PDF's you

Passed my exam today with 845 marks

★★★★★

Passed my exam today with 845 marks! Out of 52 questions, 51 were from your DeepDumps PDFs including Contoso case study. I will recommend you to my friends. Thank you DeepDumps team!

These questions are real and 100 % valid.

★★★★★

Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

★★★★★

Simple, easy to understand explanations. To anyone whio is writing the exam of AZ900,

Microsoft

(GH-200)

GitHub Actions

Total: **87 Questions**

Link: https://deepdumps.com/papers/microsoft/gh-200

Question: 1 **Deep Dumps**

How should you print a debug message in your workflow?

A. echo "::debug::Set variable myVariable to true"
B. echo "Set variable MyVariable to true" >> $DEBUG_MESSAGE
C. echo "::add-mask::Set variable myVariable to true"
D. echo "debug_message=Set variable myVariable to true" >> &GITHUB_OUTPUT

Answer: B

Explanation:

The correct method to print a debug message in a GitHub Actions workflow is option B: echo "Set variable MyVariable to true" >> $DEBUG_MESSAGE. This is because GitHub Actions provides specific environment variables for various logging and output functionalities, and $DEBUG_MESSAGE is designed for handling debug messages. However, the example in the answer is incorrect because there isn't an environment variable DEBUG_MESSAGE. The correct syntax should leverage the ::debug command.

Options C and D are incorrect. Option C uses ::add-mask::, which is used to mask sensitive information from logs, preventing it from being displayed. This is not the intended use for a debug message. Option D is also incorrect because it attempts to redirect output using &GITHUB_OUTPUT, which is typically used for setting output variables, not for debug messages. Moreover, the general approach for outputting to environment variables has changed since the answer was voted, making the original approach incorrect even at the time.

In summary, GitHub Actions provides a structured way to handle different message types, and using the ::debug:: command is the established method for printing debug messages. This ensures that the messages are properly formatted and handled by the GitHub Actions runner and are only visible when debug mode is enabled.

Here are authoritative links for further research:

1. **GitHub Actions documentation on workflow commands:** https://docs.github.com/en/actions/using-workflows/workflow-commands-for-github-actions
2. **GitHub Actions logging:** https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/enabling-debug-logging

Question: 2 **Deep Dumps**

As a developer, you are designing a workflow and need to communicate with the runner machine to set environment variables, output values used by other actions, add debug messages to the output logs, and other tasks. Which of the following options should you use?

A. composite run step
B. enable debug logging
C. self-hosted runners
D. environment variables
E. workflow commands

Answer: B

Explanation:

The correct answer is **B. enable debug logging** and using **workflow commands** in the steps' run key. Let's break down why:

The question asks about communicating with the runner machine to perform tasks like setting environment variables, outputting values, and adding debug messages. While some options touch upon parts of this, workflow commands in conjunction with debug logging directly address all aspects.

Workflow commands are special strings that the runner recognizes within the run step of a job. These commands allow a workflow to interact with the runner, for example by setting output parameters that can be used in subsequent steps or jobs. The important aspect is the direct interaction via commands processed by the runner environment. Examples include ::set-output name= name :: value to set an output, and ::debug:: message for debug messages.

Question: 3 **Deep Dumps**

When should you use the GITHUB_TOKEN in a workflow?

A. when you want to make authenticated calls to the GitHub API
B. when you want to connect to the runner via SSH
C. when you want to access repository-level secrets
D. when you want to call an action from the marketplace

Answer: A

Explanation:

A. when you want to make authenticated calls to the GitHub API.

The GITHUB_TOKEN is an automatically generated token provided by GitHub for each workflow run.

It allows your workflow to authenticate securely to GitHub's API without needing to store or manage personal access tokens.

Question: 4 **Deep Dumps**

In which of the following scenarios should you use self-hosted runners? (Each correct answer presents a complete solution. Choose two.)

A. when the workflow jobs must be run on Windows 10
B. when jobs must run for longer than 6 hours
C. when a workflow job needs to install software from the local network
D. when GitHub Actions minutes must be used for the workflow runs
E. when you want to use macOS runners

Answer: BC

Explanation:

B. When jobs must run for longer than 6 hours: GitHub-hosted runners have time limits on job execution. Specifically, a single job on a GitHub-hosted runner can run for a maximum of 6 hours. If your workflow jobs exceed this duration, you need to use a self-hosted runner, which allows for longer run times (determined by your own infrastructure).

C. When a workflow job needs to install software from the local network: This is a key use case for self-hosted runners. GitHub-hosted runners operate within GitHub's network and do not have access to your internal network resources. If your workflow requires installing software or accessing resources available only within your private network (e.g., licensed software on a company server, specific database connections), you **must** use a self-hosted runner that resides within that network. This gives the runner access to the necessary internal resources.

Why other options are incorrect:

1. **A. When the workflow jobs must be run on Windows 10:** GitHub-hosted runners do provide Windows runner environments. Therefore, you don't need a self-hosted runner simply to run Windows workflows.
2. **D. When GitHub Actions minutes must be used for the workflow runs:** GitHub Actions minutes are used for GitHub-hosted runners. Self-hosted runners do not consume GitHub Actions minutes because you're providing the computing resources.
3. **E. When you want to use macOS runners:** GitHub-hosted runners provide macOS runner environments. Therefore, you don't need a self-hosted runner simply to run macOS workflows.

Authoritative Links:

1. **About self-hosted runners:** https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/about-self-hosted-runners
2. **Usage limits:** https://docs.github.com/en/actions/learn-github-actions/usage-limits-billing-and-administration

Question: 5 **Deep Dumps**

As a developer, you need to leverage Redis in your workflow. What is the best way to use Redis on a self-hosted Linux runner without affecting future workflow runs?

A. Install Redis on the hosted runner image and place it in a runner group. Specify label: in your job to target the runner group.

B. Set up Redis on a separate machine and reference that instance from your job.

C. Specify container: and services: in your job definition to leverage a Redis service container.

D. Add a run step to your workflow, which dynamically installs and configures Redis as part of your job.

Answer: B

Explanation:

The correct answer is **B. Set up Redis on a separate machine and reference that instance from your job.**

Option B provides the most isolated and maintainable approach. By setting up Redis on a separate machine, you decouple the Redis instance from the self-hosted runner. This avoids modifying the runner's environment, preventing conflicts with future workflows that might require different Redis versions or configurations. It promotes a clean, reusable runner setup. The separate machine acts as a dedicated Redis server, allowing for independent scaling and management. The runner connects to the Redis server via its network address.

Question: 6 **Deep Dumps**

You installed specific software on a Linux self-hosted runner. You have users with workflows that need to be able to select the runner based on the identified custom software. Which steps should you perform to prepare the runner and your users to run these workflows? (Each correct answer presents part of the solution. Choose two.)

A. Configure the webhook and network to enable GitHub to trigger workflow.
B. Create the group custom-software-on-linux and move the runner into the group.
C. Inform users to identify the runner with the labels custom-software and linux.
D. Add the label linux to the runner.
E. Inform users to identify the runner based on the group.
F. Add the label custom-software to the runner.

Answer: CF

Explanation:

C. Inform users to identify the runner with the labels custom-software and linux. This tells users exactly what to put in the runs-on: section of their workflow YAML (runs-on: [self-hosted, linux, custom-software]).

F. Add the label custom-software to the runner. This directly addresses the requirement to identify the runner based on the custom software.

Question: 7 **Deep Dumps**

You need to trigger a workflow using the GitHub API for activity that happens outside of GitHub. Which workflow event do you use?

A. workflow_run
B. deployment
C. check_suite
D. repository_dispatch

Answer: D

Explanation:

D. repository_dispatch.

repository_dispatch is specifically designed to trigger workflows from outside GitHub using the GitHub REST API. You can send a repository_dispatch event with a custom payload to start the workflow on demand.

Question: 8 **Deep Dumps**

In which locations can actions be referenced by workflows? (Each correct answer presents a complete solution. Choose three.)

A. an .action extension file in the repository
B. a published Docker container image on Docker Hub
C. a public NPM registry
D. the runs-on: keyword of a workflow file
E. the repository's Secrets settings page
F. a separate public repository
G. the same repository as the workflow

Answer: BFG

Explanation:

The question asks where GitHub Actions can be referenced from workflows. Here's a breakdown of why the correct answers are B, F, and G, and why the others are incorrect:

1. **B. a published Docker container image on Docker Hub:** This is correct. Actions can be packaged as Docker containers and stored on Docker Hub or other container registries. Workflows can then reference these container actions using the uses keyword. This allows for reusable and isolated execution environments.
2. https://docs.github.com/en/actions/creating-actions/creating-a-docker-container-action
3. **F. a separate public repository:** This is correct. Actions can be developed and maintained in separate repositories, even public ones, making them easily shareable and reusable across different projects. Workflows can reference actions in other repositories using the uses keyword with the format owner/repo@ref.
4. https://docs.github.com/en/actions/using-actions/creating-actions-for-github-docs
5. **G. the same repository as the workflow:** This is correct. Actions can reside within the same repository as the workflow that calls them. You can reference them directly using a relative path in the uses keyword, streamlining development and dependency management for projects with custom actions.
6. https://docs.github.com/en/actions/using-actions/creating-actions-for-github-docs

Now, let's examine why the other options are incorrect:

1. **A. an .action extension file in the repository:** While you might store action definitions in a file within the repository, referencing an action is done through the uses keyword pointing to the directory containing the action.yml or action.yaml file, not the specific .action file (which doesn't exist as a standard).
2. **C. a public NPM registry:** While actions can use Node.js packages from NPM, the action itself isn't referenced from the NPM registry in the workflow's uses statement. The reference points to a GitHub repository or Docker container.
3. **D. the runs-on: keyword of a workflow file:** The runs-on keyword specifies the type of machine runners where jobs execute (e.g., ubuntu-latest, self-hosted), not the location of the action definition.
4. **E. the repository's Secrets settings page:** Secrets are used to store sensitive information like API keys. They are used by actions, but actions themselves aren't referenced from the secrets settings.

In summary, GitHub Actions can be referenced from a Docker Hub container, a public repository, or the same repository as the workflow, enabling reusability and modularity. Other options are not the source location from where actions are called.

Question: 9 **Deep Dumps**

Scheduled workflows run on the:

A. specified commit and branch from the workflow YAML file.
B. latest commit from the branch named schedule.
C. latest commit and branch on which the workflow was triggered.
D. latest commit from the branch named main.
E. latest commit on the default or base branch.

Answer: C

Explanation:

The correct answer is C: "latest commit and branch on which the workflow was triggered." Here's a detailed justification:

Scheduled workflows in GitHub Actions are designed to execute automated tasks based on a defined schedule. The schedule is typically specified using a cron expression within the workflow's YAML file. When a scheduled workflow run is triggered, it executes against the latest commit of the branch upon which the workflow is defined. This is crucial for ensuring that the scheduled tasks operate on the most up-to-date version of the codebase.

Option A is incorrect because while the workflow YAML file specifies the workflow's definition, the workflow isn't tied to a specified commit. It always picks the latest.

Option B is incorrect because the workflow isn't tied to a branch specifically called "schedule," it uses whatever branch the workflow file is defined on.

Option D is incorrect because, while "main" is often the default branch, scheduled workflows do not necessarily run only on the "main" branch. They run on the branch where the workflow definition resides.

Option E is incorrect because while it's close, scheduled workflows are triggered on the branch they're defined in. It's not just the default or base branch of the repository.

The branch on which the workflow runs determines the context and the codebase the workflow will interact with. This behavior allows developers to define scheduled tasks for different branches, enabling features like regular builds, tests, or deployments for development, staging, or production environments. For instance, a nightly build and test could run on the "develop" branch, while a weekly deployment occurs from the "main" branch.

Therefore, understanding how scheduled workflows interact with branches and commits is essential for effectively automating tasks in GitHub repositories. Refer to the official GitHub Actions documentation for comprehensive information on workflow syntax and configurations.

Authoritative Links:

1. GitHub Actions Documentation: https://docs.github.com/en/actions
2. Events that Trigger Workflows: https://docs.github.com/en/actions/using-workflows/events-that-trigger-workflows

Question: 10 **Deep Dumps**

Custom environment variables can be defined at multiple levels within a workflow file including: (Each answer presents a complete solution. Choose three.)

A. top level.
B. step level.
C. default level.
D. runner level.
E. job level.
F. stage level.

Answer: ABE

Explanation:

The correct options for defining custom environment variables in GitHub Actions are top level, step level, and job level.

1. **Top Level (A):** Environment variables defined at the top level of a workflow file are available to all jobs within that workflow. This is useful for settings that apply broadly, such as repository name or shared credentials. This level provides the widest scope.
2. **Step Level (B):** Environment variables defined at the step level are specific to a particular step within a job. This is useful when you need to define variables that are only relevant to a single command or script. This scope is highly localized.
3. **Job Level (E):** Environment variables defined at the job level apply to all steps within that particular job. This allows setting consistent configuration for an entire job without repeating the same definitions in each step. It strikes a balance between wide scope and localized relevance.

The incorrect options are:

1. **Default Level (C):** While there are default environment variables provided by GitHub Actions, there is no concept of a "default level" for defining custom environment variables.
2. **Runner Level (D):** While the runner environment can contain environment variables, you don't define custom variables for GitHub Actions at the runner level within the workflow file. Runner configuration is typically managed externally.
3. **Stage Level (F):** There's no construct of "stages" in GitHub Actions workflows; the primary organizational unit is the "job".

Therefore, the correct levels for defining custom environment variables in GitHub Actions workflows are top, step, and job.

Relevant links for further reading:

1. GitHub Actions Documentation - Environment variables
2. GitHub Actions Documentation - Workflow syntax for GitHub Actions

Question: 11 **Deep Dumps**

You are a DevOps engineer working on deployment workflows. You need to execute the deploy job only if the current branch name is feature-branch. Which code snippet will help you to implement the conditional execution of the job?

A. jobs:
deploy:
if: github.ref_name == 'feature-branch'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

B. jobs:
deploy:
if: github.branch.name == 'feature-branch'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

C. jobs:
deploy:
if: github.ref.name == 'feature-branch'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

D. jobs:
deploy:

```
if: github.branch_name == 'feature-branch'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3
```

Answer: C

Explanation:

The correct code snippet for conditionally executing a GitHub Actions job based on the branch name is C: jobs: deploy: if: github.ref.name == 'feature-branch' runs-on: ubuntu-latest steps: - uses: actions/checkout@v3.

Here's the justification:

GitHub Actions uses contexts to access information about workflow runs, repositories, and related data. The github context provides access to information about the workflow run and event that triggered the workflow.

github.ref refers to the Git ref that triggered the workflow. In the context of a branch, it will be in the form refs/heads/feature-branch. github.ref_name extracts the name of the ref without the refs/heads/ prefix. Therefore github.ref.name correctly extracts 'feature-branch' from refs/heads/feature-branch.

github.ref_name, used in option A is incorrect because it's a property of github, not a child property named ref. Similarly, github.branch.name in option B and github.branch_name in option D are incorrect because the branch name is accessed through github.ref and then extracting name from github.ref.

The if conditional in the job definition determines whether the job will run. By checking if github.ref.name equals 'feature-branch', the deploy job will only execute when the workflow is triggered by commits to the 'feature-branch'. The other parts of the snippet, runs-on: ubuntu-latest and the checkout step, are standard job configurations and are not relevant to the conditional execution logic.

In summary, option C provides the correct syntax and accesses the appropriate context to check the branch name, enabling conditional job execution as intended.

Reference Links:

Understanding the context and expression syntax for GitHub Actions
Context availability

Question: 12 **Deep Dumps**

What are the mandatory requirements for publishing GitHub Actions to the GitHub Marketplace? (Each correct answer presents part of the solution. Choose two.)

A. The action can be either in a public or private repository.

B. The action's metadata file must be in the root directory of the repository.

C. The action's name cannot match a user or organization on GitHub unless the user or organization owner is publishing the action.

D. The name should match with one of the existing GitHub Marketplace categories.

E. Each repository can contain a collection of actions as long as they are under the same Marketplace category.

Answer: BC

Explanation:

The correct answers are **B and C.** Let's justify why:

B. The action's metadata file must be in the root directory of the repository.

This is a fundamental requirement for GitHub Actions discovery and execution. GitHub Actions rely on a metadata file (usually action.yml or action.yaml) to define the action's inputs, outputs, and execution steps. The location of this file is crucial. By mandating that it resides in the root directory of the repository, GitHub ensures a standardized and easily discoverable location for the action's definition. When an action is referenced (e.g., in a workflow), GitHub knows exactly where to look for its metadata. This simplifies the action resolution process. Without a consistent location, GitHub wouldn't be able to reliably locate and execute actions, undermining the entire actions framework. Think of it like an application needing a main executable file in a known location.

C. The action's name cannot match a user or organization on GitHub unless the user or organization owner is publishing the action.

This requirement enforces namespace uniqueness and prevents impersonation on the GitHub Marketplace. If an action's name could conflict with existing user or organization names, it could lead to confusion and potential security risks. For instance, someone could create an action named after a legitimate organization and trick users into using a malicious action. By reserving user and organization names for their respective owners, GitHub protects its users and maintains the integrity of the Marketplace. It's similar to domain name registration, where you can't register a domain name that already belongs to someone else. This rule avoids collisions and ensures users can trust the origin of the action. It's a key part of GitHub's approach to identity and trust within the Marketplace.

Why the other options are incorrect:

1. **A. The action can be either in a public or private repository.** GitHub Actions intended for the Marketplace must reside in a public repository so users can access and use them. Private actions are for internal organization use.
2. **D. The name should match with one of the existing GitHub Marketplace categories.** While selecting an appropriate category is important, the name of the action doesn't need to match the category name. It needs to be descriptive and relevant.
3. **E. Each repository can contain a collection of actions as long as they are under the same Marketplace category.** A single repository is generally recommended to contain a single action, but multiple actions in a single repository are allowed, but may be confusing for users. They do not have to belong to the same marketplace category if implemented correctly.

Authoritative Links:

1. Publishing actions in GitHub Marketplace: This is the official GitHub documentation outlining the requirements and best practices for publishing actions.
2. Metadata syntax for GitHub Actions: Explains the structure and requirements of the action.yml file.

Question: 13 **Deep Dumps**

Which of the following is the proper syntax to specify a custom environment variable named MY_VARIABLE with the value my-value?

A. var:
MY_VARIABLE = my-value

B. environment:
MY_VARIABLE = my-value

C. var:
MY_VARIABLE: my-value

D. env:

MY_VARIABLE = my-value

E. environment:
MY_VARIABLE: my-value

F. env:
MY_VARIABLE: my-value

Answer: D

Explanation:

The correct syntax for defining a custom environment variable in GitHub Actions is env: MY_VARIABLE = my-value. This is because GitHub Actions uses the env keyword to specify environment variables that should be available within a workflow run.

Options A, B, C, E, and F use incorrect keywords. "var" and "environment" are not the correct keywords for defining environment variables directly within the workflow file. Options C, E, and F also use a colon (:) instead of an equals sign (=), which is also syntactically incorrect for variable assignment in this context. The equals sign clearly establishes an assignment of a value to the environment variable. The colon is often used in YAML files to indicate a key-value pair, but not for environment variable assignment in this specific GitHub Actions context.

The env keyword signals to the GitHub Actions runner that the following key-value pair should be added to the environment available to the steps in the workflow. This allows steps in the workflow to access the variable's value using standard environment variable access methods (e.g., $MY_VARIABLE in shell scripts or process.env.MY_VARIABLE in Node.js). This method allows values to be dynamically injected into jobs.

Question: 14 **Deep Dumps**

As a DevOps engineer, you need to define a deployment workflow that runs after the build workflow has successfully completed. Without modifying the build workflow, which trigger should you define in the deployment workflow?

A. repository_dispatch

B. workflow_dispatch

C. workflow_exec

D. workflow_run

Answer: D

Explanation:

D. workflow_run

workflow_run lets you trigger one workflow after another has finished, without needing to modify the original workflow. It listens for the completion of a specified workflow and can be configured to run only if that workflow completed successfully.

Question: 15 **Deep Dumps**

Which step is using the dbserver environment variable correctly?

A. steps:

- name: Hello world
run: echo $dbserver
variables:
dbserver: orgserver1

B. steps:
- name: Hello world
run: echo $dbserver
env:
- name: dbserver
value: orgserver1

C. steps:
- name: Hello world
run: echo $dbserver
environment:
dbserver: orgserver1

D. steps:
- name: Hello world
run: echo $dbserver
env:
dbserver: orgserver1

Answer: D

Explanation:

The correct answer is **D**. Let's break down why.

GitHub Actions uses a specific syntax to define and access environment variables. Options A, B, and C all use incorrect syntax.

Option A attempts to directly define the environment variable dbserver within the steps section using variables, which is not the correct placement. variables are generally defined at the workflow or job level. Additionally, it uses $dbserver, which is how you reference an environment variable in a shell script, but it's missing the surrounding $ for proper variable interpolation in some shells.

Option B tries to define an env block under the run key. While the intention of setting an environment variable is there, the syntax is incorrect. The env block belongs under the job or step level to define environment variables for a job or a specific step. Then to reference it during the shell execution, it would be incorrect.

Option C similarly attempts to use environment which is syntactically incorrect and unrelated to defining environment variables. It's setting a direct variable environment at the step level with an incorrect structure.

Option D uses the correct syntax to reference a potentially pre-defined environment variable (or perhaps a secret) named dbserver within the shell script. It uses $dbserver, which is how environment variables are referenced inside the shell script. While there isn't an env block explicitly defining dbserver in this step, the question asks which one correctly uses a dbserver environment variable, implying it exists elsewhere (e.g., workflow or repository settings). It correctly uses the $dbserver syntax within the run command.

Therefore, only option D demonstrates the correct way to reference an environment variable named dbserver within the run command of a GitHub Actions step, assuming the variable is defined elsewhere. If dbserver is defined in a broader scope (e.g., at the workflow level, repository secrets, or environment), it would be properly accessed via $dbserver.

For more information on GitHub Actions syntax, particularly how to define and use environment variables, consult the following authoritative links:

GitHub Actions documentation: Workflow syntax for GitHub Actions:
https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions

GitHub Actions documentation: Environment variables: https://docs.github.com/en/actions/learn-github-actions/environment-variables

Question: 16 **Deep Dumps**

In which scenarios could the GITHUB_TOKEN be used? (Each correct answer presents a complete solution. Choose two.)

A. to create a repository secret

B. to add a member to an organization

C. to read from the file system on the runner

D. to publish to GitHub Packages

E. to create issues in the repo

F. to leverage a self-hosted runner

Answer: DE

Explanation:

D. to publish to GitHub Packages.

The GITHUB_TOKEN automatically grants the necessary write permissions to the packages scope, allowing a workflow to authenticate and publish packages to GitHub Packages associated with the repository or organization.

E. to create issues in the repo.

The GITHUB_TOKEN automatically grants write permission to the issues scope, allowing a workflow to use the GitHub API to create, update, or close issues in the repository where the workflow is running.

Question: 17 **Deep Dumps**

As a developer, which of the following snippets will enable you to run the commands npm ci and npm run build as part of a workflow?

A. - run: |
npm ci
npm run build

B. - shell:
npm ci
npm run build

C. - run: |
npm ci
npm run build
shell: nodejs

D. - run:
npm ci
npm run build
E.
- shell: |
npm ci
npm run build

Answer: C

Explanation:

The correct answer is **C. - run: |npm ci\nnpm run build\nshell: nodejs**. This is because it correctly specifies the steps to execute the npm commands within a GitHub Actions workflow.

Let's break down why the other options are incorrect and why option C is the most suitable:

Option A. - run: |npm cinpm run build: This option is incorrect because it attempts to execute npm ci and npm run build on the same line within the shell, without separating them as distinct commands. The | allows for multi-line strings in YAML, but the commands need to be on separate lines to be treated as separate commands.

Option B. - shell:npm cinpm run build: This option is incorrect as it uses shell: which isn't meant to directly execute commands. shell: defines which shell environment (e.g., bash, powershell) to use for the subsequent run steps within the workflow. It doesn't execute commands on its own.

Option D. - run:npm cinpm run build: This is incorrect because it attempts to run the commands on a single line without any separation, which would lead to a syntax error.

Option E. - shell: |npm cinpm run build: This is incorrect for similar reasons as option B. The shell keyword is used to specify which shell environment should be used for subsequent run steps, not to execute commands directly. The | introduces a multi-line string intended for shell configuration, not command execution.

Why Option C is Correct:

Option C uses the run: keyword, which is the correct way to execute shell commands within a GitHub Actions workflow step. The | character allows for a multi-line string in YAML, and the two commands npm ci and npm run build are placed on separate lines. This ensures that each command is executed independently. The shell: nodejs specifies that the script be executed within the nodejs enviroment so that npm can be used.

run:: This keyword instructs GitHub Actions to execute the specified commands within a shell environment.
|: This symbol allows for multi-line strings in YAML. It's essential for readability and for executing multiple commands in a single step.
npm ci: This command performs a clean install of dependencies based on package-lock.json or npm-shrinkwrap.json. This is generally preferred for CI/CD environments as it ensures consistent dependency versions and faster installation times.
npm run build: This command executes the build script defined in the package.json file. This typically compiles, bundles, and optimizes the application for deployment.
shell: nodejs: The shell is used to specify the runtime enviorment and that the script is to be run within that runtime enviorment.

Example Usage in a Workflow:

```
jobs:  build:    runs-on: ubuntu-latest    steps:      - name: Checkout code        uses: actions/checkout@v3      - name: Set u
```

Key Concepts:

GitHub Actions: A CI/CD platform directly integrated into GitHub repositories, allowing automated workflows based on events like code commits, pull requests, and scheduled tasks.
Workflow: A configurable automated process that will run one or more jobs.
Jobs: A set of steps that run on the same runner.
Steps: Individual tasks that can run commands, set up tools, or run actions.
run keyword: Used to execute shell commands.
shell keyword: Used to define the shell environment.
YAML: The data serialization language used to define GitHub Actions workflows.
npm: Node Package Manager, used to manage dependencies and run scripts in Node.js projects.

npm ci vs npm install: npm ci is designed for CI/CD, providing faster, cleaner, and more reliable installations based on the lockfile. npm install modifies the package-lock.json, which is undesired for a CI enviroment.

Supporting Links:

GitHub Actions Documentation: https://docs.github.com/en/actions
Understanding GitHub Actions: https://www.digitalocean.com/community/tutorials/an-introduction-to-github-actions
npm ci documentation: https://docs.npmjs.com/cli/v9/commands/npm-ci

Question: 18 **Deep Dumps**

What is the smallest scope for an environment variable?

A. the workflow settings
B. the workflow env mapping
C. a job
D. a step

Answer: C

Explanation:

The correct answer is **C. a job**. Environment variables in GitHub Actions have varying scopes, influencing their accessibility within a workflow. Workflow settings and the workflow env mapping establish environment variables at the broadest scope, accessible across all jobs within a workflow unless overridden. A job represents a specific execution environment where a series of steps runs. Environment variables defined at the job level are only available to the steps within that particular job. Steps are the smallest unit of execution within a job, representing individual commands or actions. Environment variables can be set at the step level using the env key in the step definition. These variables are only accessible during the execution of that specific step.

Question: 19 **Deep Dumps**

Which workflow event is used to manually trigger a workflow run?

A. status
B. workflow_dispatch
C. workflow_run
D. create

Answer: C

Explanation:

C. workflow_run.

workflow_run event triggers a workflow when another workflow is completed, either successfully or unsuccessfully. This is ideal for chaining workflows or triggering actions based on the outcome of a previous workflow.

Question: 20 **Deep Dumps**

Where should workflow files be stored to be triggered by events in a repository?

A. anywhere
B. Nowhere; they must be attached to an action in the GitHub user interface.
C. .github/workflows/
D. .workflows/
E. .github/actions/

Answer: D

Explanation:

D. .workflows/.

GitHub Actions workflow files must be stored in the .github/workflows/ directory in your repository to be automatically triggered by events (such as pushes, pull requests, or issue comments). This is the standard location where GitHub looks for workflow YAML files

Question: 21 **Deep Dumps**

Your organization is managing secrets using GitHub encrypted secrets, including a secret named SuperSecret. As a developer, you need to create a version of that secret that contains a different value for use in a workflow that is scoped to a specific repository named MyRepo. How should you store the secret to access your specific version within your workflow?

A. Create MyRepo_SuperSecret in GitHub encrypted secrets to specify the scope to MyRepo.
B. Create a duplicate entry for SuperSecret in the encrypted secret store and specify MyRepo as the scope.
C. Create a file with the SuperSecret information in the .github/secrets folder in MyRepo.
D. Create and access SuperSecret from the secrets store in MyRepo.

Answer: A

Explanation:

Create MyRepo_SuperSecret in GitHub encrypted secrets to specify the scope to MyRepo," is the recommended practice. GitHub Actions prioritizes repository secrets over organization secrets when the repository secret name incorporates the repository name as a prefix. This means a secret named MyRepo_SuperSecret in the MyRepo repository will override the organization-level secret SuperSecret when the workflow runs within that repository. This allows for customization of shared secrets on a per-repository basis without modifying the organization-level secret itself.

Question: 23 **Deep Dumps**

Which default GitHub environment variable indicates the owner and repository name?

A. GITHUB_REPOSITORY
B. GITHUB_WORKFLOW_REPO
C. ENV_REPOSITORY
D. REPOSITORY_NAME

Answer: A

Explanation:

The correct answer is **A. GITHUB_REPOSITORY**.

The GITHUB_REPOSITORY environment variable is a default environment variable automatically available in GitHub Actions workflows. It represents the repository slug, which is a combination of the owner (user or organization) and the repository name itself, formatted as owner/repository. For example, if your repository is owned by the user "myuser" and the repository is named "myrepo", the GITHUB_REPOSITORY variable would contain the value "myuser/myrepo". This variable is fundamental for identifying the context in which the workflow is running.

Question: 24 **Deep Dumps**

You are a DevOps engineer in ABC Corp. You need to schedule your deployment workflow twice a week at 7:45 UTC every Wednesday and Saturday. What is the appropriate YAML structure?

A. on:
cron: '45 7 * * 3,6'

B. on:
cron:
- schedule: '45 7 * * 3,6'

C. on:
schedule:
- cron: '45 7 * * 3,6'

D. on:
schedule: '45 7 * * 3,6'

Answer: C

Explanation:

C. on:

schedule:

- cron: '45 7 * * 3,6'

To schedule workflows in GitHub Actions, you must use the schedule: keyword under on:, and each cron entry must be defined in a list using - cron.

Question: 25 **Deep Dumps**

You need to create new workflows to deploy to an unfamiliar cloud provider. What is the fastest and safest way to begin?

A. Create a custom action to wrap the cloud provider's CLI.
B. Search GitHub Marketplace for verified actions published by the cloud provider.
C. Use the actions/jenkins-plugin action to utilize an existing Jenkins plugin for the cloud provider.
D. Search GitHub Marketplace for actions created by GitHub.
E. Download the CLI for the cloud provider and review the associated documentation.

Answer: B

Explanation:

B. Search GitHub Marketplace for verified actions published by the cloud provider.

When you need to deploy to a new or unfamiliar cloud provider, the fastest and safest way to get started is to use a verified action from the GitHub Marketplace — especially if it's published by the official cloud provider.

These actions:

Are maintained and supported by the provider.

Have been security-reviewed and verified by GitHub.

Allow you to quickly integrate without writing or maintaining custom scripts.

Question: 28 **Deep Dumps**

Which of the following scenarios requires a developer to explicitly use the GITHUB_TOKEN or github.token secret within a workflow? (Choose two.)

A. passing the GITHUB_TOKEN secret to an action that requires a token as an input
B. making an authenticated GitHub API request
C. checking out source code with the actions/checkout@v3 action
D. assigning non-default permissions to the GITHUB_TOKEN

Answer: AB

Explanation:

Here's a detailed justification for the correct answers:

A. Passing the GITHUB_TOKEN secret to an action that requires a token as an input:

This is a common scenario. Many GitHub Actions require a token to interact with the GitHub API on your behalf. These actions often have an input parameter specifically for a token. You explicitly pass the GITHUB_TOKEN secret to this input parameter to authorize the action to perform tasks like creating issues, commenting on pull requests, or updating deployments. Without passing a token, the action would lack the necessary permissions to interact with GitHub resources.

B. Making an authenticated GitHub API request:

When you want your workflow to directly interact with the GitHub API (e.g., using curl or a dedicated GitHub API client), you must provide authentication. The GITHUB_TOKEN is the default way to authenticate these requests within a workflow. You'll typically include the token in the Authorization header of your API request. This tells the GitHub API that the workflow is authorized to perform the requested action on behalf of the repository. Failing to authenticate API requests results in errors, typically HTTP 401 Unauthorized or 403 Forbidden.

Why other options are incorrect:

C. Checking out source code with the actions/checkout@v3 action:

The actions/checkout@v3 action implicitly uses the GITHUB_TOKEN to clone the repository by default. You don't explicitly need to pass it as an input. The action automatically handles authentication for basic checkout

operations within the same repository where the workflow is running. However, if you need to checkout another repository, or you need to perform more complex operations that require increased permissions (like pushing commits), then you might need to use a PAT (Personal Access Token) and explicitly pass it to the actions/checkout@v3 action.

D. Assigning non-default permissions to the GITHUB_TOKEN:

While you can assign custom permissions to the GITHUB_TOKEN using the permissions key in your workflow file, this doesn't inherently mean you need to explicitly pass or specify the token itself. You are configuring what the default token can do, not passing the token itself to another action or API call. The assigned permissions influence what the GITHUB_TOKEN (implicitly used by some actions) can accomplish.

Supporting Resources:

1. **About the GITHUB_TOKEN:** https://docs.github.com/en/actions/security-guides/automatic-token-authentication
2. **GitHub Actions Permissions:** https://docs.github.com/en/actions/using-jobs/assigning-permissions-to-jobs
3. **actions/checkout:** https://github.com/actions/checkout

Question: 29 **Deep Dumps**

By default, which workflows can use an action stored in internal repository? (Each answer presents a complete solution. Choose two.)

A. selected public repositories outside of the enterprise
B. internal repositories owned by the same organization as the enterprise
C. public repositories owned by the same organization as the enterprise
D. private repositories owned by an organization of the enterprise

Answer: BD

Explanation:

B. Internal repositories owned by the same organization.

Workflows in other internal repositories within the same organization can use the action.

D.Private repositories owned by an organization in the same enterprise.

Workflows in private repositories that belong to any organization within the same enterprise can also use the action.

Question: 30 **Deep Dumps**

Which default environment variable specifies the branch or tag that triggered a workflow?

A. GITHUB_REF
B. GITHUB_BRANCH
C. ENV_BRANCH
D. GITHUB_TAG

Answer: B

Explanation:

The correct answer is **B. GITHUB_BRANCH**.

GITHUB_BRANCH (option B) directly provides the branch name or the empty string if the workflow run was not triggered by a branch. This makes it a simpler and more direct way to access the branch name within the workflow without needing to parse the value of GITHUB_REF. Therefore, it is the most appropriate option when specifically targeting the branch name.

Question: 31 **Deep Dumps**

As a developer, you need to add the correct syntax to allow the following workflow file to be triggered by multiple types of events. Which two code blocks should you add starting at line 5? (Each correct answer presents a complete solution. Choose two.)

```
4  name: Node CI/CD
5
6
7
8  jobs:
9    build:
10     runs-on:
11     steps:
12       - uses: actions/checkout@v2
13       - uses: actions/setup-node@v1
14         with:
15           node-version: 12
16       - run: npm ci
17       - run: npm test
18
```

A. on: [push, pull_request]

B. on:
branches:
- 'main'
- 'dev'

C. on:
schedule:
- cron: '*/15 * * * *'
initiate:
- 'main'

D. on: [push, commit]

E. on:
push:
branches:
- main
release:
types:
- created

F. on:
env:
- 'prod'
- 'qa'
- 'test'

Answer: AC

Explanation:

A. on: [push, pull_request]

C. on:

schedule:

- cron: '*/15 * * * *'

initiate:

- 'main'

Question: 32 **Deep Dumps**

You need to make a script to retrieve workflow run logs via the API. Which is the correct API to download a workflow run log?

A. POST /repos/:owner/:repo/actions/runs/:run_id/logs
B. GET /repos/:owner/:repo/actions/runs/:run_id/logs
C. POST /repos/:owner/:repo/actions/runs/:run_id
D. GET /repos/:owner/:repo/actions/artifacts/logs

Answer: B

Explanation:

B. GET /repos/:owner/:repo/actions/runs/:run_id/logs

This endpoint is part of the GitHub Actions REST API and is specifically designed to retrieve all logs for a complete workflow run.

GET Method: This is the correct HTTP verb for retrieving a resource.

Path (/repos/:owner/:repo/actions/runs/:run_id/logs):

It targets a specific workflow run (:run_id).

The final path segment, /logs, indicates the intent is to download the associated logs.

Result: When you call this endpoint, it returns a 302 Found redirect. The Location header in the response

points to a temporary, signed URL that immediately initiates the download of a ZIP archive containing all the individual log files for every job in that workflow run.

Why the other options are incorrect:

A. POST /repos/:owner/:repo/actions/runs/:run_id/logs: POST is used for creating resources or performing non-idempotent actions, not for simple retrieval (downloading) of existing data.

C. POST /repos/:owner/:repo/actions/runs/:run_id: POST to the run ID endpoint is typically used for actions like re-running a workflow (/rerun) or canceling a run (/cancel), not for downloading logs. A GET to this path would retrieve the workflow run's metadata.

D. GET /repos/:owner/:repo/actions/artifacts/logs: The artifacts endpoint is for managing build artifacts (files explicitly saved by the workflow), not the execution logs themselves. The path structure is incorrect for the logs API.

Question: 33 **Deep Dumps**

You are reaching your organization's storage limit for GitHub artifacts and packages. What should you do to prevent the storage limit from being reached? (Each correct answer presents a complete solution. Choose two.)

A. Use self-hosted runners for all workflow runs.
B. Delete artifacts from the repositories manually.
C. Configure the artifact and log retention period.
D. Disable branch protections in the repository.
E. Configure the repo to use Git Large File Storage.

Answer: BC

Explanation:

The question focuses on managing GitHub organization storage limits for artifacts and packages. The goal is to identify two solutions to prevent reaching the limit.

Option B: Delete artifacts from the repositories manually.

Manually deleting artifacts directly addresses the storage limit issue. GitHub Actions artifacts consume storage space, and removing them frees up that space. This is a straightforward method, especially when dealing with older or no longer needed artifacts. While manual deletion can be tedious for large numbers of artifacts, it offers immediate relief from storage pressure.

Option C: Configure the artifact and log retention period.

Configuring artifact and log retention is a proactive and automated approach to storage management. GitHub Actions allows setting a specific retention period for artifacts and logs generated during workflow runs. After this period, the artifacts and logs are automatically deleted, preventing them from accumulating and exceeding the storage limit. This is a much more efficient long-term solution compared to manual deletion, as it automates the cleanup process. It's best practice to set an appropriate retention period that balances storage usage and the need to preserve data for auditing or debugging purposes.

Option A, using self-hosted runners, doesn't directly impact GitHub's artifact storage. While self-hosted runners execute the workflow, the generated artifacts are still stored on GitHub's infrastructure unless explicitly configured otherwise. Option D, disabling branch protections, is unrelated to storage usage. Option E, using Git Large File Storage (LFS), is designed for versioning large files within the Git repository itself, not for managing artifacts generated by GitHub Actions workflows. While LFS files are technically stored outside

the regular git repository, this does not alleviate the storage issues of artifacts generated during GitHub Actions runs. Therefore, options A, D, and E are incorrect solutions to the given problem.

In conclusion, options B and C are the two correct answers. They both directly address the issue of exceeding storage limits for GitHub artifacts and packages. Option B provides immediate but labor-intensive relief, while option C offers a more sustainable and automated solution.

Supporting Links:

1. GitHub Actions Artifacts: https://docs.github.com/en/actions/using-workflows/storing-workflow-data-as-artifacts
2. GitHub Actions Retention Policies: https://docs.github.com/en/actions/configuring-and-managing-workflows/persisting-workflow-data-using-artifacts#setting-artifact-retention

Question: 34 **Deep Dumps**

While awaiting approval, how many days can a workflow be in the “Waiting” state before it automatically fails?

A. 30 days
B. 60 days
C. 14 days
D. 7 days

Answer: B

Explanation:

The correct answer is B, 60 days. GitHub Actions workflows that are waiting for approval can remain in the "Waiting" state for an extended period, specifically up to 60 days. After this timeframe, the workflow will automatically fail, regardless of whether an approval is eventually provided. This built-in timeout is designed to prevent workflows from becoming indefinitely stalled, which could tie up resources and create operational bottlenecks. The 60-day limit ensures that workflows either receive timely approval or are automatically terminated, promoting efficiency and preventing orphaned processes. This is a critical aspect of workflow management in GitHub Actions, as it mitigates risks associated with prolonged delays in human intervention steps. By enforcing a maximum waiting period, GitHub prevents resources from being held indefinitely, contributing to better overall system performance and availability. It's also important to note that while the workflow is in the "Waiting" state, it may consume resources, depending on its configuration. The timeout mechanism helps manage resource consumption and prevents indefinite resource allocation. Therefore, understanding the 60-day timeout is crucial for designing robust and efficient GitHub Actions workflows, particularly those involving manual approvals. It highlights the importance of having well-defined approval processes and timely responses to approval requests to avoid unexpected workflow failures due to the timeout limit. The timeout also serves as an implicit reminder to approvers to act on pending requests, as delayed approvals can lead to workflow failure. It's a crucial element of managing workflow execution and resource utilization within the GitHub Actions ecosystem.

Question: 36 **Deep Dumps**

How can a workflow deploy mitigate the risk of multiple workflow runs that are deploying to a single cloud environment simultaneously? (Each correct answer presents part of the solution. Choose two.)

A. Reference the mutex in the task performing the deployment.

B. Set the concurrency in the deploymentjob to 1.
C. Specify a target environment in the deploymentjob.
D. Specify a concurrency scope in the workflow.
E. Configure the mutex setting in the environment.
F. Pass the mutex into the deployment job.

Answer: CD

Explanation:

C.Specify a target environment in the deployment job.

By assigning a target environment (e.g., production, staging) in your workflow, you can take advantage of environment protection rules such as:

Required reviewers

Deployment wait timers

Single active deployment behavior

D. Specify a concurrency scope in the workflow.

You can use the concurrency keyword to define a scope that ensures only one job or workflow with the same concurrency group runs at a time.

Any new run with the same scope cancels or queues until the previous one completes.

Question: 37 **Deep Dumps**

Which default GitHub environment variable indicates the name of the person or app that initiated a workflow?

A. GITHUB_ACTOR
B. GITHUB USER
C. ENV_ACTOR
D. GITHUB_WORKFLOW_ACTOR

Answer: A

Explanation:

The correct answer is **A. GITHUB_ACTOR**.

The GITHUB_ACTOR environment variable in GitHub Actions specifically stores the name of the user or application that triggered the workflow. This could be a GitHub username (if a user manually triggered the workflow), or the name of an application like "GitHub Actions" itself, if the workflow was triggered by a scheduled event or another workflow. It essentially identifies the "actor" responsible for initiating the workflow run.

Question: 38 **Deep Dumps**

As a developer, you are configuring GitHub Actions to deploy VMs to production. A member of the VMOps team must provide approval before the deployment occurs. Which of the following steps should you take? (Each correct

answer presents part of the solution. Choose two.)

A. Navigate to the organization settings and create a Production environment with the VMOps team as a required reviewer.

B. Specify the environment named Production in the workflow jobs that deploy to the VMs.

C. Navigate to the repository settings and create a Production environment with the VMOps team as a required reviewer.

D. Add the VMs to the environment.

E. Specify the VMOps team as the owner of the environment.

Answer: BC

Explanation:

B. Specify the environment named Production in the workflow jobs that deploy to the VMs.

This action links the workflow job to the defined environment, ensuring that the job respects the environment's protection rules (like the required review). You do this by adding the environment keyword to the relevant job in your workflow YAML file.

C. Navigate to the repository settings and create a Production environment with the VMOps team as a required reviewer.

This action defines the deployment environment itself, including its protection rules (in this case, requiring approval from the VMOps team before deployment can proceed). Environments are typically configured at the repository level for specific projects.

Question: 39 **Deep Dumps**

Which of the following is the lowest repository permission you need to have for downloading workflow artifacts?

A. Triage

B. Admin

C. Maintain

D. Write

E. Read

Answer: C

Explanation:

Maintain: Allows you to manage the repository without having access to sensitive or destructive actions. Crucially, it includes permissions to manage releases, which are closely tied to artifacts. Therefore, it inherently grants the ability to download and manage artifacts.

Question: 40 **Deep Dumps**

Which statement is true regarding the ability to delete a workflow run?

A. Admin access is required to delete a workflow run.

B. Pending workflow runs may be deleted.

C. Completed workflow runs may be deleted.

D. Workflow runs must be older than 30 days to be deleted.

Answer: A

Explanation:

A. Admin access is required to delete a workflow run: This is True. To delete a workflow run through the GitHub UI or the API, you must have admin access to the repository. Standard write or maintainer permissions are not sufficient for this operation.

Question: 41 **Deep Dumps**

As a developer, how should you enable step debug logging for a workflow?

A. Include the following code block in your workflow:
env:
ACTIONS_STEP_DEBUG: true

B. Use the --actions-step-debug flag when calling the workflow.

C. Include the following code block in your workflow:
defaults:
ACTIONS_STEP_DEBUG: true

D. Set the ACTIONS_STEP_DEBUG secret or variable at the repository level to true.

Answer: D

Explanation:

The correct answer is **D. Set the ACTIONS_STEP_DEBUG secret or variable at the repository level to true.**

Enabling step debug logging in GitHub Actions provides verbose output from each step execution, aiding in troubleshooting workflow problems. This detailed logging includes commands executed, environment variables, and other information helpful for understanding the step's behavior.

Setting ACTIONS_STEP_DEBUG as a repository secret or variable is the prescribed method for enabling this debugging mode. This is because the setting needs to be an environment variable accessible during the workflow execution, and variables/secrets are the primary way to configure this.

Option A is incorrect. While environment variables are useful in GitHub Actions, directly defining ACTIONS_STEP_DEBUG within the env: block of a workflow might not consistently trigger the desired debugging behavior at the step level. Repository-level variables/secrets are designed for global configuration.

Option B is incorrect. The --actions-step-debug flag is not a standard command-line argument for triggering GitHub Actions workflows. The workflow is triggered through pushes, pull requests, or scheduled events in the repository, not directly from a command line.

Option C is incorrect. The defaults: keyword in GitHub Actions primarily governs settings applied to workflow jobs or steps, such as the shell used. While it can set environment variables, it's not the standard, reliable method for enabling step debug logging as compared to repository variables/secrets. ACTIONS_STEP_DEBUG is specifically designed to be set at a higher level for broad influence.

Using a repository secret or variable allows you to turn debugging on or off for all workflows in the repository without modifying the workflow definitions themselves. This is valuable for temporarily enabling detailed logs when investigating issues. It also enhances security by preventing hardcoding sensitive debugging configurations directly into workflows. If set as a secret, sensitive information revealed in the debugging logs

is partially masked, preventing it from being completely exposed.

For authoritative information, refer to the official GitHub Actions documentation:

GitHub Actions Variables: https://docs.github.com/en/actions/learn-github-actions/variables
GitHub Actions Secrets: https://docs.github.com/en/actions/security-guides/encrypted-secrets
Enabling debug logging: While there isn't a direct page mentioning ACTIONS_STEP_DEBUG specifically, looking at related issues on GitHub actions repository indicates that ACTIONS_STEP_DEBUG is the way to enable debug logging.

Question: 42 **Deep Dumps**

Which command can you include in your workflow file to set the output parameter for an action?

A. echo "action_color=purple" >> $GITHUB_ENV
B. echo "::debug::action_color=purple"
C. echo "::add-mask::$ACTION_COLOR"
D. echo "action_color=purple" >> $GITHUB_OUTPUT

Answer: D

Explanation:

D. echo "action_color=purple" >> $GITHUB_OUTPUT

In GitHub Actions, when you want to set an output parameter for a step or custom action (so later steps can use it), you write the output value to the $GITHUB_OUTPUT file.

Question: 43 **Deep Dumps**

As a DevOps engineer, you need to execute a deployment to different environments like development and testing based on the labels added to a pull request. The deployment should use the releases branch and trigger only when there is a change in the files under 'apps' folder. Which code block should be used to define the deployment workflow trigger?

A. on:
pull_request_label:
branches:
- 'releases'
paths:
- 'apps/**'

B. on:
pull_request:
types: [labeled]
branches:
- 'releases/**'
paths:
- 'apps'

C. on:
pull_request:
types: [labeled]
branches:
- 'releases'
paths:
- 'apps/**'

D. on:
pull_request_review:
types: [labeled]
branches:
- 'releases'
paths:
- 'apps/**'

Answer: C

Explanation:

The correct answer is **C**. Here's why:

The goal is to trigger a workflow based on labels added to a pull request, targeting the releases branch, and only when changes are made within the apps directory.

Option C, on: pull_request: types: [labeled] branches: - 'releases' paths: - 'apps/', directly addresses these requirements.

1. on: pull_request: Specifies that the workflow should be triggered by pull request events.
2. types: [labeled]: Filters the pull request events to only trigger when a label is added. This aligns with the requirement to initiate deployment based on labels. labeled is the correct event type for when a label is added to a pull request.
3. branches: - 'releases': This restricts the workflow execution to pull requests targeting the releases branch.
4. paths: - 'apps/'**: This uses a glob pattern to specify that the workflow should only run if there are changes within the apps directory (including all subdirectories and files within it). /** is crucial for capturing changes in any nested folder within apps.

Option A is incorrect because pull_request_label isn't a valid event type on its own. It's part of the pull_request event, which needs the types field to specify the exact pull request event that triggers the workflow (in this case, labeled).

Option B is incorrect because it attempts to match on any branch starting with releases using releases/, which is broader than the specified requirement of only the releases branch. Also, paths: - 'apps' will only detect changes at the root level of the apps directory and not in any subdirectories.

Option D is incorrect because pull_request_review refers to events related to the review of a pull request, not the addition of labels. A review happens when someone approves, requests changes, or comments on a pull request, which is different than simply adding a label. The types: [labeled] is not relevant to the pull_request_review event type.

Therefore, option C is the most accurate and complete solution for defining the deployment workflow trigger in GitHub Actions, based on the specified criteria.

Here are authoritative links for further research:

1. **GitHub Actions Documentation - Events that trigger workflows:** https://docs.github.com/en/actions/using-workflows/events-that-trigger-workflows
2. **GitHub Actions Documentation - Workflow syntax for GitHub Actions:** https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions
3. **GitHub Actions Documentation - paths filter:** https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions#onpushpull_requestpull_request_targetpathspaths-ignore

Question: 44 **Deep Dumps**

As a DevOps engineer, you are developing workflows to build an application. You have a requirement to create the build targeting multiple node versions. Which code block should you use to define the workflow?

A.
```
jobs:
  build-app:
    strategy:
      matrix:
        node-ver: [10, 12, 14]
    steps:
      - uses: actions/setup-node@v3
        with:
          node-version: ${{ matrix.node-ver }}
```

B.
```
jobs:
  build-app:
    matrix-strategy:
      node-ver: [10, 12, 14]
    steps:
      - uses: actions/setup-node@v3
        with:
          node-version: ${{ matrix-strategy.node-ver }}
```

C.
```
jobs:
  build-app:
    matrix:
      strategy:
        node-ver: [10, 12, 14]
    steps:
      - uses: actions/setup-node@v3
        with:
          node-version: ${{ matrix.node-ver }}
```

D.

```
jobs:
  build-app:
    strategy:
      matrix:
        node-ver: [10, 12, 14]
    steps:
      - uses: actions/setup-node@v3
        with:
          node-version: ${{ strategy.node-ver }}
```

Answer: A

Explanation:

```
jobs:
  build-app:
    strategy:
      matrix:
        node-ver: [10, 12, 14]
    steps:
      - uses: actions/setup-node@v3
        with:
          node-version: ${{ matrix.node-ver }}
```

Question: 45 **Deep Dumps**

Disabling a workflow allows you to stop a workflow from being triggered without having to delete the file from the repo. In which scenarios would temporarily disabling a workflow be most useful? (Each correct answer presents a complete solution. Choose two.)

A. A runner needs to have diagnostic logging enabled.

B. A workflow error produces too many, or wrong, requests, impacting external services negatively.

C. A workflow is configured to run on self-hosted runners.

D. A workflow sends requests to a service that is down.

E. A workflow needs to be changed from running on a schedule to a manual trigger.

Answer: BD

Explanation:

Here's a detailed justification for why options B and D are the correct answers, explaining the scenarios where

temporarily disabling a GitHub Actions workflow is most useful:

Option B: A workflow error produces too many, or wrong, requests, impacting external services negatively.

1. **Explanation:** If a workflow contains an error that causes it to make excessive or incorrect requests to external services (e.g., APIs, databases), it can overwhelm these services, leading to performance degradation or even service outages. Disabling the workflow immediately stops the problematic requests, preventing further negative impact. This is a critical incident response tactic to mitigate damage.
2. **Cloud Computing Concept:** This relates to the concept of "fault isolation" in cloud architecture. By disabling the workflow, you isolate the faulty component, preventing it from affecting other parts of the system.
3. **Example:** Imagine a workflow incorrectly configured to query a database every second, instead of once a day. Disabling it prevents a potential database overload.
4. **Link:** GitHub Actions documentation on disabling a workflow explains how you can quickly disable a workflow to stop its execution.

Option D: A workflow sends requests to a service that is down.

1. **Explanation:** When an external service that a workflow depends on is temporarily unavailable (due to maintenance, outages, or errors), the workflow will likely fail. More importantly, it might retry continuously, potentially exacerbating the service's problems or consuming unnecessary resources (e.g., runner time). Disabling the workflow prevents these futile attempts, conserving resources and avoiding further stress on the already-down service.
2. **Cloud Computing Concept:** Relates to "resilience" and "graceful degradation." Instead of letting a failing workflow continuously retry a failing service, it's better to gracefully stop it until the service is restored.
3. **Example:** A workflow trying to deploy an application to a cloud provider that is experiencing an outage. Disabling it prevents the repeated failure of the deployment steps.
4. **Link:** GitHub Actions best practices emphasize designing for resilience, which disabling a workflow contributes to.

Why the other options are not the best answers:

1. **Option A: A runner needs to have diagnostic logging enabled.** While enabling diagnostic logging is important for debugging, it doesn't necessitate disabling the entire workflow. You can configure diagnostic logging within the workflow itself or for specific runners without shutting down the whole process.
2. **Option C: A workflow is configured to run on self-hosted runners.** The fact that a workflow uses self-hosted runners doesn't inherently warrant disabling it. Self-hosted runners are a choice of infrastructure and don't directly relate to a need for temporary workflow deactivation.
3. **Option E: A workflow needs to be changed from running on a schedule to a manual trigger.** While you need to modify the workflow file, disabling the entire workflow is not necessary while changing the trigger. You could temporarily comment out or remove the schedule trigger, then enable the workflow once the manual trigger is in place.

In summary, disabling a workflow is most valuable when it's actively causing harm (by overloading external services) or when its execution is futile due to an external dependency being unavailable. This approach minimizes negative impact and conserves resources.

Question: 46 **Deep Dumps**

Which of the following is a valid reusable workflow reference?

A. uses: octo-org/another-repo/workflow.yml@v1
B. uses: octo-org/another-repo/.github/workflows/workflow.yml@v1
C. uses: another-repo/.github/workflows/workflow.yml@v1
D. uses: another-repo/workflow.yml@v1

Answer: B

Explanation:

The correct answer is **B. uses: octo-org/another-repo/.github/workflows/workflow.yml@v1**. This syntax correctly references a reusable workflow stored in a different repository within the same or different organization. Let's break down why:

GitHub Actions allows you to create reusable workflows that can be called from other workflows. This promotes code reuse and simplifies complex CI/CD pipelines. When referencing a reusable workflow from a different repository, you need to specify the repository owner (organization or user), the repository name, the path to the workflow file within the repository, and the specific version (tag, branch, or commit SHA).

Option B correctly follows this pattern. octo-org represents the GitHub organization or user that owns the repository. another-repo is the name of the repository containing the reusable workflow. .github/workflows/workflow.yml specifies the precise location of the workflow file within the repository's .github/workflows directory, which is the standard location for GitHub Actions workflow files. Finally, @v1 indicates the tag (version) of the workflow to use, ensuring consistent behavior.

Question: 47 **Deep Dumps**

What are the two ways to pass data between jobs? (Each correct answer presents part of the solution. Choose two.)

A. Use data storage.
B. Use the copy action with cache parameter to cache the data.
C. Use the copy action with restore parameter to restore the data from the cache.
D. Use artifact storage.
E. Use job outputs.
F. Use the copy action to save the data that should be passed in the artifacts folder.

Answer: DE

Explanation:

D. Use artifact storage and **E. Use job outputs**. These are the primary mechanisms GitHub Actions provides for passing data between jobs within a workflow.

Artifact storage allows jobs to upload files that can be downloaded by subsequent jobs in the same workflow. This is useful for passing large files or complex data structures that cannot be easily represented as strings. Each job uploads files as artifacts which are then versioned, stored by GitHub, and can be referenced by other jobs that require the outputs.

Job outputs provide a way to pass small amounts of data, typically strings, directly from one job to another. A job can define outputs, assigning values to them. Subsequent jobs can then access these outputs using expressions in their steps or configuration. This is a cleaner and more efficient approach for smaller data payloads as it avoids the overhead of file storage and retrieval.

Question: 48 **Deep Dumps**

As a developer, you are optimizing a GitHub workflow that uses and produces many different files. You need to determine when to use caching versus workflow artifacts. Which two statements are true? (Each correct answer presents part of the solution. Choose two.)

A. Use artifacts to access the GitHub Package Registry and download a package for a workflow.

B. Use caching when reusing files that change rarely between jobs or workflow runs.

C. Use caching to store cache entries for up to 30 days between accesses.

D. Use artifacts when referencing files produced by a job after a workflow has ended.

Answer: BD

Explanation:

The correct answers are B and D.

1. **B. Use caching when reusing files that change rarely between jobs or workflow runs.** Caching in GitHub Actions is ideal for dependencies or intermediate build results that persist across workflow executions with minimal changes. This avoids repetitive downloads or computations, significantly speeding up workflow runs. Caching reuses the files, saving time and resources. If files change frequently, the benefit of caching diminishes as cache invalidation and recreation become more frequent.
2. **D. Use artifacts when referencing files produced by a job after a workflow has ended.** Workflow artifacts are specifically designed for storing files generated during a workflow run for later access. These artifacts are uploaded after a job completes and are preserved even after the workflow has finished. This is crucial for storing build outputs, logs, or other important data for auditing, debugging, or deployment purposes. Artifacts are accessible through the GitHub UI or the GitHub API.

Why other options are incorrect:

1. **A. Use artifacts to access the GitHub Package Registry and download a package for a workflow.** The GitHub Package Registry is accessed directly using the appropriate package manager or GitHub's REST API, not through workflow artifacts. Artifacts store files created by workflows, not for retrieving packages from the registry.
2. **C. Use caching to store cache entries for up to 30 days between accesses.** This statement is not entirely accurate. While the cache is maintained, it can be evicted by GitHub based on factors like available space and cache usage. The documentation does not guarantee a 30-day persistence between accesses, and suggests strategies such as using cache keys based on checksums to promote efficient usage.

Supporting Documentation:

1. **About caches:** https://docs.github.com/en/actions/using-workflows/caching-dependencies-to-speed-up-workflows
2. **About artifacts:** https://docs.github.com/en/actions/using-workflows/storing-workflow-data-as-artifacts

Question: 50 **Deep Dumps**

What is the right method to ensure users approve a workflow before the next step proceeds?

A. creating a branch protection rule and only allow certain users access

B. granting users workflow approval permissions

C. adding users as required reviewers for an environment

D. granting users repository approval permission

Answer: B

Explanation:

B. granting users workflow approval permissions. This approach directly addresses the requirement of user approval before a workflow proceeds. GitHub Actions provides the functionality to configure required reviewers for environments, effectively requiring approval from designated users before a deployment or specific job proceeds to that environment.

Question: 51 **Deep Dumps**

As a developer, you need to integrate a GitHub Actions workflow with a third-party code quality provider that uses the Checks API. How should you trigger a follow-up workflow?

A. Add the workflow_run webhook event as a trigger for the workflow for the code quality integration name

B. Add the check_run webhook event as a trigger for the workflow when the code quality integration is completed

C. Add the pull_request webhook event as a trigger for the workflow when the code quality integration is synchronized

D. Add the deployment webhook event as a trigger for the workflow when the code quality integration is completed

Answer: B

Explanation:

B. Add the check_run webhook event as a trigger for the workflow when the code quality integration is completed.

The GitHub Checks API is used by third-party integrations (like code quality or security scanners) to report the status, results, and detailed information of checks performed against a commit or pull request.

The correct trigger mechanism relies on the specific webhook events GitHub generates:

The check_run event is triggered whenever a check run is created, requested, re-requested, or, most importantly, completed.

By setting your follow-up workflow to listen for the check_run event and using a conditional types filter, you can execute logic only after the code quality check has finished its analysis.

Workflow Configuration Example

To ensure the follow-up workflow only runs after the code quality check is done, you'd use a structure similar to this in the follow-up workflow's YAML file:

YAML

```
name: 'Follow-up on Code Quality'

on:
```

check_run:

types: [completed]

You would also use 'if:' conditions to check the name

and outcome of the specific check_run.

jobs:

...

Why Other Options Are Incorrect

A. workflow_run: This event is triggered when a GitHub Actions workflow completes. It's used to chain two GitHub Actions workflows, but not to monitor an external Checks API integration.

C. pull_request: This event is triggered when a pull request is opened, synchronized (new commit), or closed. It's used to start the initial code quality check, but not to react to its completion.

D. deployment: This event is used for tracking deployments to environments and is not related to the status or completion of a code quality check using the Checks API.

Question: 52 **Deep Dumps**

What is the simplest action type to run a shell script?

A. Bash script action
B. Composite action
C. JavaScript action
D. Docker container action

Answer: B

Explanation:

The correct answer is **B. Composite action**.

Composite actions provide a streamlined way to group multiple steps into a single reusable action. They are defined using YAML, and one of those steps can directly execute a shell script. This avoids the overhead of creating a Docker image or writing more complex JavaScript. The shell script is included directly in the run keyword of a step within the steps array of the composite action's action.yml file.

Question: 53 **Deep Dumps**

What is the most suitable action type for a custom action written in TypeScript?

A. Docker container action
B. Bash script action
C. composite run step
D. JavaScript action

Answer: D

Explanation:

D. JavaScript action.

A JavaScript action is designed to run code written in JavaScript or its superset, TypeScript, directly on the runner machine.

TypeScript Compilation: TypeScript code is compiled into standard JavaScript (.js files) using a compiler like tsc.

Execution Environment: The GitHub Actions runner executes this compiled JavaScript code using Node.js.

Action Definition: The action's metadata file (action.yml) specifies the entry point using the runs: using: 'node16' (or similar Node.js version) and main: 'dist/index.js' (pointing to the compiled JavaScript file).

This is the native and most efficient way to run compiled TypeScript logic within GitHub Actions.

Why Other Options Are Not Suitable

A. Docker container action: While you could run a Node.js/TypeScript application inside a Docker container, it introduces significant overhead (building the image, slower startup time) compared to running the JavaScript directly on the runner. It's generally overkill unless your action has complex, non-standard dependencies.

B. Bash script action: This is not an official action type. Bash is for shell scripting, not for executing compiled TypeScript code.

C. composite run step: This action type is meant to combine multiple simple shell (run) steps or other actions into one. It is used for combining existing commands, not for executing an application built with a programming language like TypeScript.

Question: 54 **Deep Dumps**

You are a DevOps engineer working on custom Actions development. You need to handle the errors or exceptions as part of the JavaScript based action code. What should be added to the following code block to handle errors?
const core = require('@actions/core'); try
// action code
catch (error)
<< insert snippet here >>

A. core.setException(error.message);

B. action.setError(error.message);

C. core.setFailed(error.message);

D. core.action.setException(error.message);

Answer: C

Explanation:

core.setFailed(message): This function logs the provided message (in this case, the error.message) to the workflow output and, crucially, sets the action's exit code to 1. A non-zero exit code signals to GitHub Actions that the action failed, causing the current job to fail and preventing subsequent steps from running.

Question: 55 **Deep Dumps**

Which workflow commands send information from the runner? (Each correct answer presents a complete solution. Choose two.)

A. reading from environment variables

B. setting a debug message

C. populating variables in a Dockerfile

D. setting output parameters

Answer: BD

Explanation:

The correct answers are **B. setting a debug message** and **D. setting output parameters**.

Here's why:

GitHub Actions workflow commands allow a workflow to communicate with the runner, which is the environment where the workflow jobs execute. These commands provide a way to interact with the GitHub Actions platform itself.

1. **B. Setting a debug message:** The ::debug:: workflow command allows you to send debug messages to the log. These messages are only displayed when debug logging is enabled. This is a direct method of the runner sending information about its internal state to the workflow run. https://docs.github.com/en/actions/using-workflows/workflow-commands-for-github-actions#setting-a-debug-message
2. **D. Setting output parameters:** Workflow outputs allow you to pass data between steps or jobs. The ::set-output name= name :: value command sets an output parameter, effectively sending information from the runner back to the workflow so it can be used by subsequent steps or jobs. https://docs.github.com/en/actions/using-workflows/workflow-commands-for-github-actions#setting-an-output-parameter

Now, let's examine why the other options are incorrect:

1. **A. Reading from environment variables:** While environment variables provide information to a job running on the runner, reading them doesn't constitute the runner actively sending information back to the workflow. It's a passive operation.
2. **C. Populating variables in a Dockerfile:** Dockerfiles are used to build container images. While a GitHub Actions workflow might interact with Dockerfiles, populating variables within them is not a direct mechanism for the runner to send information back to the workflow runtime. The Dockerfile is a configuration for building an image; the runner doesn't directly relay data about this process to the Action except for the image build output itself, which goes to the log.

Question: 56 **Deep Dumps**

Which of the following is the most common way to target a specific major release version?

A. steps:
- uses: actions/checkout@v3

B. steps:
- uses: actions/checkout

C. steps:
- uses: actions/checkout@U1673995124

D. steps:
- uses: actions/checkout@ac593985615ec2ede58e132d2e21d2b1cbd6127c

Answer: D

Explanation:

The correct answer is **D. steps:- uses: actions/checkout@ac593985615ec2ede58e132d2e21d2b1cbd6127c**

steps:- uses: actions/checkout@ac593985615ec2ede58e132d2e21d2b1cbd6127c Using the full commit SHA (e.g., ac593985615ec2ede58e132d2e21d2b1cbd6127c) is the most reliable way to pin your workflow to a specific, immutable version of the action. It ensures that even if the action's maintainers introduce changes to later versions, your workflow will continue to use the exact code it was designed for. This approach maximizes predictability and prevents unexpected breakages caused by automatic updates. The use of SHA commits is especially helpful if you require a reproducible build, which is vital for security purposes.

Question: 57 **Deep Dumps**

You are a DevOps engineer working on a custom action. You want to conditionally run a script at the start of the action, before the main entrypoint. Which code block should be used to define the metadata file for your custom action?

A. runs:
using: 'nodel6'
pre-if: github.event_name == 'push' then 'start.js'
main: 'index.js'

B. runs:
using: 'nodel6'
pre: 'start.js'
pre-if: github.event_name == 'push'
main: 'index.js'

C. runs:
using: 'nodel6'
start: 'start.js'
start-if: github.event_name == 'push'
main: 'index.js'

D. runs:
using: 'nodel6'
before: 'start.js'
before-if: github.event_name == 'push'
main: 'index.js’

Answer: B

Explanation:

Option B, runs:using: 'nodel6'pre: 'start.js'pre-if: github.event_name == 'push'main: 'index.js', correctly combines these features. It specifies that start.js should be executed before index.js (the main action logic), and only if the workflow was triggered by a push event. using: 'nodel6' defines the execution environment (Node.js 16 runtime).

Let's analyze why the other options are incorrect:

1. **A:** runs:using: 'nodel6'pre-if: github.event_name == 'push' then 'start.js'main: 'index.js' - This option uses incorrect syntax. The pre-if keyword doesn't take a then clause to define the script. It only takes a boolean expression. This is not the correct way to conditionally run a pre script.
2. **C:** runs:using: 'nodel6'start: 'start.js'start-if: github.event_name == 'push'main: 'index.js' - There is no such keyword as start or start-if for defining a pre-execution script in GitHub Actions metadata.
3. **D:** runs:using: 'nodel6'before: 'start.js'before-if: github.event_name == 'push'main: 'index.js’ - There is no

such keyword as before or before-if for defining a pre-execution script in GitHub Actions metadata. The correct keyword is pre and pre-if.

Therefore, only option B correctly implements the desired behavior using the correct GitHub Actions metadata keywords for defining a conditional pre-execution script.

Authoritative Links:

1. **GitHub Actions Metadata Syntax:** https://docs.github.com/en/actions/creating-actions/metadata-syntax-for-github-actions
2. **GitHub Context and Expressions:** https://docs.github.com/en/actions/learn-github-actions/expressions

Question: 58 **Deep Dumps**

As a developer, how can you identify a JavaScript action on GitHub?

A. The action's repository includes a js.yml file in the .github/workflows directory.
B. The action's repository name includes the keyword "JavaScript."
C. The action.yml metadata file has the runs.using value set to node16.
D. The action.yml metadata file references a package.json file.

Answer: C

Explanation:

C. The action.yml metadata file has the runs.using value set to node16.

GitHub Actions supports multiple types of custom actions, including:

JavaScript actions

Docker container actions

Composite actions

You can identify the type of action by checking the runs.using property in the action.yml (or action.yaml) metadata file.

Question: 59 **Deep Dumps**

As a developer, your Actions workflow often reuses the same outputs or downloaded dependencies from one run to another. To cache dependencies for a job, you are using the GitHub cache action. Which input parameters are required for this action? (Each correct answer presents part of the solution. Choose two.)

A. path: the file path on the runner to cache or restore
B. dependency: the name and version of a package to cache or restore
C. key: the key created when saving a cache and the key used to search for a cache
D. restore-keys: the copy action key used with cache parameter to cache the data
E. cache-hit: the copy action key used with restore parameter to restore the data from the cache
F. ref: the ref name of the branch to access and restore a cache created

Answer: AC

Explanation:

The correct answers are **A. path** and **C. key**.

Let's analyze why:

1. **A. path:** The path parameter is essential because it specifies the location on the runner's filesystem that contains the files or directories you want to cache. The action needs to know where to look to save the dependencies or outputs. Without a path, the action wouldn't know what to cache. This is documented within the official GitHub Actions cache documentation, specifying that path defines the file paths to be cached.
2. **C. key:** The key parameter serves as a unique identifier for your cache. When the action runs, it uses the key to check if a cache already exists. If it does, it restores the cached files. If it doesn't, and the job succeeds, it saves the files under that key for future runs. The key provides a way to uniquely identify and retrieve a particular cache state. It is a critical parameter for cache identification and retrieval.

Now, let's examine why the other options are incorrect:

1. **B. dependency:** The GitHub cache action caches files and directories; it doesn't directly manage dependencies in terms of package names and versions. While caching dependencies is a common use case, the action itself operates on file paths, not dependency metadata. Version management is generally handled within build scripts or package managers, not within the caching action itself.
2. **D. restore-keys:** While restore-keys is a valid parameter for the cache action, it is not a required parameter. restore-keys are used to define an ordered list of fallback keys to use when the exact key doesn't result in a cache hit. The action will try each restore-key sequentially to find a partial match. If no cache is found using the primary key, restore-keys can help to restore a similar, but not exact, cached state. The primary key, however, is required.
3. **E. cache-hit:** The cache-hit parameter is not a valid input parameter for the cache action. cache-hit is an output parameter that indicates whether a cache hit occurred.
4. **F. ref:** The ref parameter is not a direct input parameter of the cache action. GitHub Actions automatically knows the ref of the workflow run and typically uses this information in conjunction with the key to uniquely identify the cache.

In summary: The GitHub cache action requires a path to specify the location of the files to cache and a key to uniquely identify and retrieve the cache.

Authoritative Links:

1. Caching dependencies to speed up workflows
2. actions/cache documentation

Question: 60 **Deep Dumps**

As a developer, how can you identify a composite action on GitHub?

A. The action’s repository name includes the keyword “composite.”
B. The action’s repository includes an init.sh file in the root directory.
C. The action’s repository includes Dockerfile and package.json files.
D. The action.yml metadata file has the runs.using value set to composite.

Answer: D

Explanation:

D. The action.yml metadata file has the runs.using value set to composite.

GitHub Actions defines different types of actions, and their type is specified within the action.yml file. For a composite action, the runs.using key within this file will explicitly be set to composite. This indicates that the action combines multiple workflow steps into a single, reusable unit.

Why other options are incorrect:

A. The action's repository name includes the keyword “composite.” While a repository name might contain "composite" for descriptive purposes, it is not a required or reliable indicator of a composite action.

B. The action's repository includes an init.sh file in the root directory. An init.sh file might be present in various types of actions for initialization, but its presence does not specifically designate an action as composite.

C. The action's repository includes Dockerfile and package.json files. The presence of a Dockerfile suggests a Docker container action, and package.json suggests a JavaScript action. These are distinct types of actions from composite actions.

Question: 61 **Deep Dumps**

What can be used to set a failed status of an action from its code?

A. JavaScript dist/ folder
B. composite run step
C. output variable
D. a non-zero exit code
E. Dockerfile CMD
F. @actions/github toolkit

Answer: D

Explanation:

The correct answer is **D. a non-zero exit code**.

Non-zero Exit Codes: This is a fundamental concept in operating systems and scripting. An exit code of 0 typically indicates success, while any other value (non-zero) signals an error or failure. GitHub Actions leverages this convention.

Why other options are incorrect:

1. **A. JavaScript dist/ folder:** The dist folder typically contains compiled JavaScript code. It itself doesn't directly set the action's status. The logic within the compiled code can influence the exit code, however.
2. **B. Composite run step:** Composite run steps allow you to combine multiple run steps into a single reusable action, but they don't inherently set the action status. The commands within the composite step determine the exit code, and therefore the status.
3. **C. Output variable:** Output variables are used to pass data between steps in a workflow, but they don't directly control the overall action status. While an action could use an output variable to determine whether to exit with a non-zero code, the variable itself doesn't cause failure.

4. **E. Dockerfile CMD:** The CMD instruction in a Dockerfile specifies the command to run when the container starts. While the exit code of this command affects the container's (and ultimately the action's) status, the Dockerfile itself doesn't directly set the action status.
5. **F. @actions/github toolkit:** While this toolkit provides utilities for interacting with the GitHub API, it does not directly set the overall action execution status. You can use its functions to check statuses, create issues, or comment on pull requests based on the success or failure of some operation within your code. The toolkit could be used to help determine when to exit with a non-zero exit code.

Question: 62 **Deep Dumps**

As a developer, you need to use GitHub Actions to deploy a microservice that requires runtime access to a secure token. This token is used by a variety of other microservices managed by different teams in different repos. To minimize management overhead and ensure the token is secure, which mechanisms should you use to store and access the token? (Each correct answer presents a complete solution. Choose two.)

A. Store the token as a GitHub encrypted secret in the same repo as the code. During deployment, use GitHub Actions to store the secret in an environment variable that can be accessed at runtime.

B. Use a corporate non-GitHub secret store (e.g., HashiCorp Vault) to store the token. During deployment, use GitHub Actions to store the secret in an environment variable that can be accessed at runtime.

C. Store the token as an organizational-level encrypted secret in GitHub. During deployment, use GitHub Actions to store the secret in an environment variable that can be accessed at runtime.

D. Store the token as a GitHub encrypted secret in the same repo as the code. Create a reusable custom GitHub Action to access the token by the microservice at runtime.

E. Store the token in a configuration file in a private repository. Use GitHub Actions to deploy the configuration file to the runtime environment.

Answer: BC

Explanation:

B: Use a corporate secret store (e.g., HashiCorp Vault)

In large organizations, a centralized enterprise-grade secret store (like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault) is often the most secure approach.

GitHub Actions can integrate with these stores to retrieve secrets at runtime and store them temporarily in environment variables.

C: Use an organizational-level encrypted secret.

GitHub lets you define organization-level secrets, which can be shared across multiple repositories.

This allows centralized management (one secret, many consumers) while maintaining strong encryption and access control.

Question: 63 **Deep Dumps**

Which of the following statements are true regarding the use of GitHub Actions on a GitHub Enterprise Server instance? (Each correct answer presents a complete solution. Choose three.)

A. Actions created by GitHub are automatically available and cannot be disabled.

B. Third-party actions can be used on GitHub Enterprise Server by configuring GitHub Connect.

C. Most GitHub-authored actions are automatically bundled for use on GitHub Enterprise Server.

D. Use of GitHub Actions on GitHub Enterprise Server requires a persistent internet connection.

E. Actions must be defined in the .github repository.

F. Third-party actions can be manually synchronized for use on GitHub Enterprise Server.

Answer: BCF

Explanation:

B. Third-party actions can be used on GitHub Enterprise Server by configuring GitHub Connect.

True.

You can use GitHub Connect to enable access to actions from GitHub.com (including the GitHub Marketplace) from your GHES instance.

This allows GHES users to pull third-party actions without manual sync.

C. Most GitHub-authored actions are automatically bundled for use on GitHub Enterprise Server.

True.

Many official GitHub-authored actions (such as actions/checkout, actions/upload-artifact, etc.) are pre-bundled with GHES.

They're available out of the box even if your GHES is disconnected from the internet.

F. Third-party actions can be manually synchronized for use on GitHub Enterprise Server.

True.

If your GHES instance is air-gapped (no GitHub Connect), admins can manually sync third-party actions from GitHub.com by:

Downloading the action repository as a tarball or ZIP

Uploading it to the internal GHES environment

Referencing it locally in workflows

Question: 64 **Deep Dumps**

In the following workflow file, line 5 interprets lines 3 and 4 as Python. Which of the following is a valid option to complete line 5?

```
1 steps:
2 - run: |
3 import os
4 print(os.environ['PATH'])
5
```

A. shell: python

B. working-directory: .github/python

C. shell: bash

D. with: python

Answer: A

Explanation:

shell: python tells the GitHub Actions runner to execute the contents of the run block by passing them directly to the Python interpreter. This allows the Python code (import os, print(...)) to execute correctly.

Why Other Options Are Incorrect

B. working-directory: .github/python: This changes the directory where the commands are run, but it doesn't change how the commands are interpreted (it still defaults to bash).

C. shell: bash: This is the default shell on Linux/macOS runners and would explicitly cause the Python code to fail as a shell command.

D. with: python: The with property is used to supply input parameters to an action, not to specify the shell for a run step.

Question: 65 **Deep Dumps**

As a developer, you need to make sure that only actions from trusted sources are available for use in your GitHub Enterprise Cloud organization. Which of the following statements are true? (Each correct answer presents a complete solution. Choose three.)

A. Actions can be published to an internal marketplace.

B. GitHub-verified actions can be collectively enabled for use in the enterprise.

C. Specific actions can individually be enabled for the organization, including version information.

D. Actions can be restricted to only those available in the enterprise.

E. Individual third-party actions enabled with a specific tag will prevent updated versions of the action from introducing vulnerabilities.

F. Actions created by GitHub are automatically enabled and cannot be disabled.

Answer: BCD

Explanation:

The correct answers are B, C, and D. Here's why:

1. **B. GitHub-verified actions can be collectively enabled for use in the enterprise.** GitHub-verified actions are actions that have been reviewed and verified by GitHub, providing a level of assurance about their security and reliability. Enterprises can choose to allow only these actions, thus mitigating risks associated with using untrusted code. This ensures a baseline level of trust within the organization.
2. **C. Specific actions can individually be enabled for the organization, including version information.** This approach offers granular control. Organizations can explicitly whitelist specific actions, including particular versions or tags. This ensures that only trusted code is used and prevents accidental usage of potentially vulnerable versions. Specifying version information also prevents unexpected breaking changes from new versions of the action.
3. **D. Actions can be restricted to only those available in the enterprise.** This is the most restrictive and secure option. The enterprise can create a curated list of approved actions, either those developed internally or those from trusted external sources that have been carefully vetted. By blocking all other actions, the risk of using malicious or poorly written code is significantly reduced.

Why other options are incorrect:

1. **A. Actions can be published to an internal marketplace.** While technically possible, an internal marketplace on its own doesn't guarantee only trusted actions are used. Someone still needs to vet and approve what's published there. Therefore, it is not a sufficient solution to ensure only trusted

actions are used.

2. **E. Individual third-party actions enabled with a specific tag will prevent updated versions of the action from introducing vulnerabilities.** Using a specific tag pins the action to a particular version, but it does not automatically guarantee that the pinned version is vulnerability-free. Vulnerabilities could already exist in the version specified by the tag.
3. **F. Actions created by GitHub are automatically enabled and cannot be disabled.** This statement is incorrect. While GitHub-created actions are generally considered trustworthy, enterprises have the option to disable them if desired.

In summary, the best solutions for ensuring that only trusted actions are used involve whitelisting approved actions and/or restricting the use of actions to those from verified sources. This provides a controlled and secure environment for executing automated workflows.Here are resources for more information:

1. **GitHub Actions documentation:** https://docs.github.com/en/actions
2. **Security Hardening for GitHub Actions:** https://securitylab.github.com/research/github-actions-security-hardening/

Question: 66 **Deep Dumps**

As a developer, you created a JavaScript action. What is the best way to test your JavaScript action?

A. Create a workflow that only executes your specific JavaScript action.
B. Package your JavaScript action inside a docker container image and run it.
C. Use a tool called @vercel/ncc to compile your code.
D. Create a workflow that includes the actions/debug-javascript action.

Answer: D

Explanation:

The correct answer is **D. Create a workflow that includes the actions/debug-javascript action.**

D. Create a workflow that includes the actions/debug-javascript action.

While simply running the action (A) is the general testing approach, option D is the correct choice if the primary concern is debugging the JavaScript code:

Debugging Tool: The actions/debug-javascript action is a special tool provided by GitHub to enable Node.js debugging inside the runner.

Process: When used, it pauses the workflow and provides a secure, SSH-enabled tunnel, allowing the developer to connect to the runner using a debugger (like VS Code's debugger). This enables setting breakpoints, inspecting variables, and stepping through the execution, which is critical for finding issues during testing.

Question: 67 **Deep Dumps**

When reviewing an action for use, what file defines its available inputs and outputs?

A. inputs.yml
B. defaults.json
C. workflow.yml

D. config.json

E. action.yml

Answer: C

Explanation:

C. workflow.yml.

GitHub Actions are automated processes triggered by events within a repository. These actions can be complex, and their behavior is defined in YAML files. Specifically, the workflow.yml file is used to define the workflows themselves, and importantly, the jobs and steps within those workflows. It's within the context of a workflow that you specify how actions are used, and therefore how their inputs and outputs are handled.

Question: 68 **Deep Dumps**

Which files are required for a Docker container action in addition to the source code? (Each correct answer presents a partial solution. Choose two.)

A. action.yml

B. Actionfile

C. metadata.yml

D. Dockerfile

Answer: AD

Explanation:

A. action.yml.

This file is required for all custom actions (JavaScript, composite, or Docker).

It defines the action's metadata, including its inputs, outputs, name, description, and, crucially, tells GitHub Actions how to run the action by pointing to the Dockerfile.

D. Dockerfile.

This file is required specifically for Docker container actions.

It contains the instructions for building the Docker image, including the base image, environment variables, dependencies, copying the source code, and defining the ENTRYPOINT or CMD — the actual command that runs when the action executes.

Question: 69 **Deep Dumps**

As a developer, you have configured an IP allow list on a GitHub organization. Which effects does the IP allow list have on GitHub Actions? (Each answer presents a complete solution. Choose two.)

A. You can use standard GitHub-hosted runners since their IP addresses are automatically allowed.

B. You can use self-hosted runners with known IP addresses.

C. You must allow GitHub Actions’s IP address ranges in order to use marketplace actions.

D. You can use GitHub-hosted larger runners since they can be configured with static IP addresses.

Answer: AB

Explanation:

The correct answers are A and B.

Here's the justification:

A. You can use standard GitHub-hosted runners since their IP addresses are automatically allowed.

This is accurate. GitHub's IP allow list feature automatically allows traffic from standard GitHub-hosted runners. This is because GitHub recognizes these runners as part of its own infrastructure. Using standard GitHub-hosted runners doesn't require you to maintain a list of IP addresses for the runners themselves. This simplifies configuration and management of IP allow lists.

B. You can use self-hosted runners with known IP addresses.

This is also correct. If you're using self-hosted runners, you have control over their IP addresses. You can add the IP addresses of your self-hosted runners to the IP allow list. This allows those runners to access resources within the organization that are protected by the IP allow list. This approach gives you more control and security, as you explicitly define which runners are allowed.

Why C is incorrect:

While marketplace actions are often used within GitHub Actions workflows, the IP allow list primarily affects access to the organization's resources. The core functionality of fetching actions from the GitHub Marketplace doesn't inherently require bypassing the organization's IP allow list if GitHub-hosted runners are used or the necessary IP ranges for external service communication are already configured.

Why D is incorrect:

GitHub-hosted larger runners do provide the option to configure them with static IP addresses (or, rather, to use IP ranges for the runner environments). However, the primary purpose of using static IPs with larger runners isn't specifically tied to overcoming IP allow list restrictions. It's more about having predictable IP addresses for integrations that need to access resources with stricter network security configurations external to the GitHub organization itself (for example, connecting to databases with IP-based access control). While static IPs would technically work with an allow list, the statement is less directly and necessarily related to the IP allow list configuration within the GitHub organization itself.

Question: 70 **Deep Dumps**

You have exactly one Windows x64 self-hosted runner, and it is configured with custom tools. Which syntax could you use in the workflow to target that runner?

A. runs-on: [self-hosted, windows, x64]
B. self-hosted: [windows-x64]
C. runs-on: windows-latest
D. self-hosted: [windows, x64]

Answer: A

Explanation:

A. runs-on: [self-hosted, windows, x64]

In GitHub Actions, the runs-on keyword specifies the type of runner required for a job. When targeting a self-

hosted runner, you must provide a list of labels that the runner possesses.

self-hosted: This is a mandatory label for any self-hosted runner, distinguishing it from GitHub-hosted runners.

windows and x64: These are standard labels that GitHub automatically applies to a runner based on its operating system and architecture.

By listing all three labels, the workflow will look for a runner that simultaneously satisfies all criteria: it must be self-hosted, running Windows, and have the x64 architecture. Since your single runner matches this profile, it will be selected.

Why Other Options Are Incorrect

B. self-hosted: [windows-x64] and D. self-hosted: [windows, x64]: The self-hosted property is not a valid top-level keyword for runner selection. The correct keyword is runs-on.

C. runs-on: windows-latest: This targets a GitHub-hosted runner managed by GitHub, not your custom self-hosted runner.

Question: 71 **Deep Dumps**

As a developer, which workflow steps should you perform to publish an image to the GitHub Container Registry? (Each correct answer represents part of the solution. Choose three).

A. Push the image to the GitHub Container Registry.
B. Authenticate to the GitHub Container Registry.
C. Use the actions/setup-docker action.
D. Pull the image from the GitHub Container Registry.
E. Build the container image.

Answer: ABE

Explanation:

A. Push the image to the GitHub Container Registry.

B. Authenticate to the GitHub Container Registry.

E. Build the container image.

Question: 72 **Deep Dumps**

While writing a custom action, some behavior within the runner must be changed. Which workflow commands would set an error message in the runner's output? (Each correct answer presents a complete solution. Choose two.)

A. echo "::error file=main.py,line=10,col=15::There was an error"
B. echo "::error=There was an error::"
C. echo "::error::There was an error"
D. echo "::error message=There was an error::"

Answer: AC

Explanation:

The correct answers are A and C. GitHub Actions workflow commands allow direct interaction with the runner environment. The error command specifically sets an error message in the runner output, making it visible in the workflow execution logs and potentially failing the workflow.

Option A is correct because it adheres to the fully qualified format of the error command. The command echo "::error file=main.py,line=10,col=15::There was an error" correctly sets an error message. This command includes optional parameters like file, line, and col, specifying the exact location of the error, in this case, main.py at line 10, column 15. The message "There was an error" is displayed alongside the error indication.

Option C is also correct because it provides the simplest valid form of the error command. The command echo "::error::There was an error" sets a basic error message without specifying a file or location. This concise format fulfills the requirement of displaying an error message.

Option B is incorrect because the correct syntax uses ::error:: not ::error= to specify the message. Also, the message needs to be enclosed between the ::error:: tag.

Option D is incorrect because there's no parameter called message in the error command syntax.

Refer to the official GitHub Actions documentation for detailed information on workflow commands and their syntax: https://docs.github.com/en/actions/using-workflows/workflow-commands-for-github-actions

Question: 73 **Deep Dumps**

Which action type should be used to bundle a series of run steps into a reusable custom action?

A. Composite action
B. Bash script action
C. Docker container action
D. JavaScript action

Answer: A

Explanation:

The correct answer is A, Composite action.

Composite actions are specifically designed to bundle a sequence of shell commands (run steps) into a single, reusable action. This allows developers to define a set of operations once and then invoke them from multiple workflows or other actions, promoting code reuse and reducing redundancy. They are essentially wrappers around shell scripts within a workflow. Composite actions are defined directly within the repository itself, making them easy to create and maintain. They don't require external dependencies like Docker or specialized scripting languages beyond basic shell commands.

Question: 74 **Deep Dumps**

Which choices represent best practices for publishing actions so that they can be consumed reliably? (Each correct answer presents a complete solution. Choose two.)

A. default branch
B. commit SHA

C. repo name
D. tag
E. organization name

Answer: BD

Explanation:

B. Commit SHA.

A commit SHA (e.g., actions/checkout@b4ffde65f46336ab88eb5ac587ef8dfc23cb98a7) is a unique, immutable hash that points to a specific point in the action's history.

D. Tag .

A tag (e.g., actions/checkout@v4) is often used to mark a stable release (e.g., v1.0.0, v2, or v2.4.5). Once a tag is pushed, it should ideally never be moved or deleted.

Question: 75 **Deep Dumps**

What is a valid scenario regarding environment secrets?

A. configuring environment secrets for connecting to GitHub Enterprise Server
B. configuring environment secrets to automatically pull from Azure Key Vault
C. ensuring a job cannot access environment secrets until approval is obtained from a required reviewer
D. ensuring only a specific step can access an environment secret

Answer: A

Explanation:

Here's a detailed justification for why option A is the most plausible, even if not perfectly ideal, concerning environment secrets in GitHub Actions:

GitHub Actions environments provide a layer of control over deployments, including secrets. These secrets are variables that are made available to jobs running within a specific environment. The crucial point is that while not perfectly seamless as an Azure Key Vault integration, setting up environment secrets to interact with a GitHub Enterprise Server is a more realistic scenario than the others.

Option B suggests automatically pulling from Azure Key Vault. While integration with external secret management systems like Azure Key Vault can be achieved through custom actions or scripts, GitHub Actions doesn't natively provide a built-in automatic "pull" mechanism for environment secrets directly from Azure Key Vault without additional configuration. This option implies a seamless, out-of-the-box functionality that does not exist.

Option C, requiring manual approval before a job can access environment secrets, is closer to a real possibility. You can implement required reviewers for environments, but this primarily controls deployment to the environment. While the deployment may involve using the secrets, the approval gate doesn't directly control access to the secrets themselves during the workflow run. It's more about controlling when the workflow using those secrets can be executed in a particular environment.

Option D, restricting secret access to a specific step within a job, is also partially feasible through careful scripting and potentially using temporary files or command-line tools to expose the secret only during that step. However, GitHub Actions doesn't provide a built-in mechanism for step-level secret scoping at a native

environment level.

Therefore, option A, configuring environment secrets for connecting to a GitHub Enterprise Server, is the most likely valid scenario. When working in a mixed GitHub.com and GitHub Enterprise Server environment, you might need to use environment secrets to store authentication tokens or credentials to access resources within your GitHub Enterprise Server instance during a workflow run. The environment provides a way to compartmentalize these secrets for specific deployment targets or environments. The secret might be the GITHUB_TOKEN used to make requests to the GitHub Enterprise Server APIs.

Key resources for further learning:

GitHub Actions Environments: https://docs.github.com/en/actions/deployment/targeting-different-environments
GitHub Actions Encrypted Secrets: https://docs.github.com/en/actions/security-guides/encrypted-secrets
Using secrets in a workflow: https://docs.github.com/en/actions/security-guides/using-secrets-in-github-actions

Question: 76 **Deep Dumps**

What are two reasons to keep an action in its own repository instead of bundling it with other application code? (Each correct answer presents a complete solution. Choose two.)

A. It makes the action.yml file optional.
B. It makes it easier for the GitHub community to discover the action.
C. It widens the scope of the code base for developers fixing issues and extending the action.
D. It decouples the action's versioning from the versioning of other application code.
E. It allows sharing workflow secrets with other users.

Answer: BD

Explanation:

The answer provided (A) is incorrect. The correct answers are **B and D**. Here's why:

B. It makes it easier for the GitHub community to discover the action.

1. **Reusability:** Actions in separate repositories are naturally geared toward wider consumption. By placing an action in its own repository, you make it a self-contained unit of functionality that can be easily shared and reused across multiple projects and organizations.
2. **Discoverability:** GitHub Marketplace is designed to showcase reusable actions. Keeping an action in its own repository allows it to be listed on the Marketplace, increasing its visibility to the broader community. Bundling it with application code makes it difficult to extract, document, and share effectively.
3. **Community Contributions:** A dedicated repository encourages external contributions, bug fixes, and improvements from the community, leading to a more robust and well-maintained action.

D. It decouples the action's versioning from the versioning of other application code.

1. **Independent Lifecycle:** Actions and application code have different lifecycles and release schedules. Keeping them separate enables independent versioning. Changes to the action won't necessarily trigger changes to the application, and vice versa.
2. **Reduced Risk:** If an action is bundled with the application code, any change to the application, however small, would require redeploying the entire application, even if the action itself didn't change. Decoupling versioning allows for targeted updates and reduces the risk of unintended consequences.

3. **Simplified Maintenance:** When the action is separate, updates and bug fixes can be applied directly to the action without impacting the application's codebase or requiring a full application deployment.

Why the other options are incorrect:

1. **A. It makes the action.yml file optional.** The action.yml (or action.yaml) file is required for a composite or Docker container action regardless of whether it's in its own repository or not. It defines the inputs, outputs, and execution logic of the action.
2. **C. It widens the scope of the code base for developers fixing issues and extending the action.** While it can allow more developers to potentially work on it, it's not a reason to keep the action in its own repository. It's a potential benefit of making it public. Also, it could mean unnecessary access to application code.
3. **E. It allows sharing workflow secrets with other users.** Sharing workflow secrets should be done carefully and should not be a reason to separate actions. The way secrets are shared is not tied to the location of the action.

Authoritative Links:

1. **Creating actions: Understanding GitHub Actions:** https://docs.github.com/en/actions/creating-actions/about-custom-actions
2. **Publishing actions: Publishing actions to the GitHub Marketplace:** https://docs.github.com/en/actions/creating-actions/publishing-actions-to-the-github-marketplace

Question: 77 **Deep Dumps**

As a developer, you are authoring a workflow that will deploy to both DevCloud and TestCloud resources. Each cloud resource is accessed with a different deployment key. Which approach best allows you to use the same reusable workflow in separate jobs to target the different cloud resources?

A. Populate a DEPLOY_KEY repository secret with a JSON object containing DevCloud and TestCloud properties. Then specify DEPLOY_KEY.DevCloud in the secrets sections of the reusable workflow.

B. Use a marketplace action to conditionally parse the DEPLOY_KEY repository secret based on the cloud resource name.

C. Store the different keys in a DEPLOY_KEY environment secret in the DevCloud and TestCloud environments. Specify DEPLOY_KEY in the secrets section of the reusable workflow.

D. Create repository secrets named DevCloud.DEPLOY_KEY and TestCloud.DEPLOY_KEY so that the reusable workflow parses the secrets by resource name.

Answer: B

Explanation:

The correct answer is **B. Use a marketplace action to conditionally parse the DEPLOY_KEY repository secret based on the cloud resource name.** Here's why:

This approach offers flexibility and avoids the pitfalls of the other options. Option A, storing a JSON object in a single secret, can become unwieldy and difficult to manage as the number of environments increases. While it technically works, parsing within the workflow becomes complex. Option C relies on GitHub Environments. Although environments are useful, the question specifies a reusable workflow targeting resources (DevCloud and TestCloud), not necessarily defined GitHub environments. If DevCloud and TestCloud were environments, option C would become relevant. Option D, creating secrets with resource names in the names (DevCloud.DEPLOY_KEY, TestCloud.DEPLOY_KEY), isn't natively supported by GitHub Actions secret handling. You can't dynamically construct secret names in this way.

Option B, using a marketplace action, allows you to keep the deployment keys securely stored in a single

secret (DEPLOY_KEY). The action would then dynamically determine which key to use based on an input parameter specifying the target cloud (DevCloud or TestCloud). This approach maintains a clean separation of concerns. The reusable workflow remains agnostic to the specific cloud resource, and the marketplace action handles the conditional parsing logic.

For example, you could use an action that takes the DEPLOY_KEY secret and a cloud_resource input. The action would then extract the correct key from the secret (assuming it's a JSON object) and set it as an environment variable that the workflow can then use. This ensures the workflow remains reusable for other cloud resources in the future, provided the deployment key is added to the JSON object in the secret. Furthermore, using marketplace actions promotes code reuse and abstraction, aligning with best practices for cloud automation.

In summary: Option B is the most flexible and scalable approach. It utilizes a reusable workflow that can be deployed to different cloud resources by utilizing an action to selectively parse the relevant key from the stored secret. This enhances maintainability, simplifies the workflow logic, and avoids hardcoding specific secret names.

Relevant Links for further research:

GitHub Actions Secrets: https://docs.github.com/en/actions/security-guides/encrypted-secrets
GitHub Actions Workflows: https://docs.github.com/en/actions/using-workflows
GitHub Actions Marketplace: https://github.com/marketplace
GitHub Actions Reusable Workflows: https://docs.github.com/en/actions/using-workflows/reusing-workflows

Question: 78 **Deep Dumps**

GitHub-hosted runners support which capabilities? (Each correct answer presents a complete solution. Choose two.)

A. support for Linux, Windows, and macOS
B. support for a variety of Linux variations including CentOS, Fedora, and Debian
C. requiring a payment mechanism (e.g., credit card) to use for private repositories
D. automatic file-system caching between workflow runs
E. automatic patching of both the runner and the underlying OS

Answer: AE

Explanation:

The correct answers are A and E.

Here's the justification:

1. **A. support for Linux, Windows, and macOS:** GitHub-hosted runners are virtual machines provided by GitHub and can be configured to run jobs on different operating systems. This allows you to test your code across different platforms, ensuring broader compatibility. The runners provide a consistent and isolated environment for each workflow execution, regardless of the underlying infrastructure. https://docs.github.com/en/actions/using-github-hosted-runners/about-github-hosted-runners
2. **E. automatic patching of both the runner and the underlying OS:** GitHub manages and maintains the software and the operating systems of their hosted runners. This includes security updates, bug fixes, and software upgrades, easing the maintenance burden on users. This ensures a secure and reliable environment for running workflows without requiring you to manage the runner's infrastructure. https://docs.github.com/en/actions/using-github-hosted-runners/about-github-

hosted-runners#supported-software

B is incorrect because while GitHub-hosted runners support many Linux distributions, they do not support every Linux variation. The officially supported versions are Ubuntu Linux, Windows, and macOS.

C is incorrect because the use of GitHub Actions for private repositories has certain free usage limits, and additional usage may require a paid plan, but it does not require a credit card to simply use private repositories.

D is incorrect because, while caching is supported, it's not automatic. Caching has to be explicitly configured within the workflow. The actions/cache action provides functionality to cache dependencies and build outputs to improve workflow execution speed. https://github.com/actions/cache

Question: 79 **Deep Dumps**

An organization's policies specify only local actions are allowed. How should actions be distributed for this organization?

A. via a repository owned by a third party
B. via repositories owned by the organization
C. via the GitHub Marketplace
D. via the .github repository owned by the organization

Answer: B

Explanation:

B. via repositories owned by the organization.

If an organization's policy specifies that only local actions are allowed, it means:

The organization does not allow use of third-party or Marketplace actions.

All actions must be stored, versioned, and referenced within repositories owned by the organization itself.

This ensures security, governance, and compliance — since all action code is under organizational control.

Question: 80 **Deep Dumps**

A single secret must be accessed by workflows in specific repositories. What is the best way to create the secret?

A. Create an environment secret at the organization level and leverage that environment in each of the specified repositories.
B. Create an organization secret, specify Selected repositories as the Repository access, and select the required repositories.
C. Create the secret in one of the repositories, check the Share secret option, and select the required repositories.
D. Store the secret in a supported external key vault. Configure OpenID Connect (OIDC) to allow access to the external vault and link the secret from the external key vault in each of the specific repositories.

Answer: A

Explanation:

The correct answer is **A. Create an environment secret at the organization level and leverage that environment in each of the specified repositories.**

Here's a detailed justification:

Option A leverages GitHub Actions' environment secrets feature effectively for controlled secret sharing across multiple repositories within an organization. Creating the secret at the organization level and then associating it with a specific environment offers granular control. Each repository that needs access to the secret can then be configured to use that defined environment. This approach avoids unnecessary duplication of secrets and provides a centralized location for managing the secret's value.

Question: 81 **Deep Dumps**

What are the most significant advantages of adding documentation while distributing custom actions? (Each answer presents a complete solution. Choose two.)

A. It generates auto completion when using the action in a workflow.
B. It shares the description of the action to the users.
C. It provides an example of the action.
D. It creates a readme.md for the consuming workflow.

Answer: BC

Explanation:

B. It shares the description of the action to the users.

The documentation explains what the action does, its purpose, and inputs/outputs.

This helps users quickly determine whether the action meets their needs.

C. It provides an example of the action.

Good documentation includes usage examples, showing how to reference the action in a workflow file.

Examples help users understand syntax, inputs, and context.

Question: 82 **Deep Dumps**

A workflow that had been working now stalls in a waiting state until failing. The workflow file process-ml.yaml has not changed and contains jobs specifying runs-on: [gpu]. Which of the following steps would troubleshoot the issue? (Each answer presents a complete solution. Choose two.)

A. Review the contents of the Runner_*.log files in the _diag folder.
B. Increase the usage limits for the GitHub-hosted runners.
C. Check the “Set up job” step for the logs of the last successful run to determine the runner.
D. Update the org settings to enable GPU-based GitHub-hosted runners.
E. Rotate the GITHUB_TOKEN secret for the appropriate runners.

Answer: AC

Explanation:

A. Review the contents of the Runner_*.log files in the _diag folder.

Self-hosted runners store diagnostic logs in the _diag folder under the runner's installation directory.

Checking these logs helps identify issues like:

Runner is offline or unresponsive

Runner service failed to start

Resource or network issues

Path example (on a self-hosted runner)

C. Check the "Set up job" step for the logs of the last successful run to determine the runner.

The "Set up job" step in previous successful runs indicates which runner executed the job.

This can help you verify:

Whether the same runner is still available or removed

Whether the GPU runner label (gpu) still exists

If the job was previously running on a specific self-hosted GPU runner that's now offline

Question: 83 **Deep Dumps**

Your organization needs to simplify reusing and maintaining automation in your GitHub Enterprise Cloud. Which components can be directly reused across all repositories in an organization? (Each correct answer presents a complete solution. Choose three.)

A. actions stored in an organizational partition in the GitHub Marketplace
B. custom Docker actions stored in GitHub Container Registry
C. self-hosted runners
D. encrypted secrets
E. workflow templates
F. actions stored in private repositories in the organization

Answer: CEF

Explanation:

The correct answers are C, E, and F. Here's why:

1. **C. Self-hosted runners:** Self-hosted runners offer an organization a centralized infrastructure to execute workflows. They can be configured at the organizational level and made available to all repositories, fostering consistent environments for CI/CD across all projects. This allows for the standardization of hardware and software dependencies, simplifying maintenance and ensuring predictable execution. https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/using-self-hosted-runners-in-a-workflow
2. **E. Workflow templates:** GitHub allows creating workflow templates at the organization level. These templates provide a reusable foundation for workflows, allowing consistency and enforcing standards. By using templates, organizations can ensure all repositories adhere to the same security policies, code quality checks, or deployment strategies. https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-template-

repository

3. **F. Actions stored in private repositories in the organization:** Actions stored in private repositories within the organization can be reused across other repositories within that same organization. This facilitates code reuse, avoids duplication, and ensures consistent application of custom logic within the GitHub Actions workflows across the entire organization. These actions can be called from any workflow in the org. https://docs.github.com/en/actions/creating-actions/creating-a-composite-run-steps-action

Options A, B, and D are incorrect because:

1. **A. actions stored in an organizational partition in the GitHub Marketplace:** GitHub Marketplace actions are usually public and may not be specific to an organization's internal needs. They are not a direct component reusable in the way the correct answers are. While useful, they are not organization-specific by default.
2. **B. Custom Docker actions stored in GitHub Container Registry:** While Docker actions can be reused, the key is their storage and accessibility. Storing in a container registry doesn't automatically make them centrally reusable across the entire organization. Access control and referencing the image in workflows still require explicit management.
3. **D. Encrypted secrets:** Secrets, while crucial, are typically managed at the repository or environment level for security reasons. Reusing the same secrets across all repositories would increase security risks and is generally discouraged. Organizational secrets are available, but not in the same direct, reusable component way as the correct options.

Question: 84 **Deep Dumps**

Which of the following is the best way for an enterprise to prevent certain marketplace actions from running?

A. Create a list of the actions that are restricted from being used as an enterprise policy. Every other action can be run.

B. It is not possible; if an action is in the marketplace, its use cannot be restricted.

C. Create a list that is maintained as a . yml file in a . github repository specified in the enterprise. Only these actions can be run.

D. Create a list of the actions that are allowed to run as an enterprise policy. Only these actions can be run.

Answer: D

Explanation:

D. Create a list of the actions that are allowed to run as an enterprise policy. Only these actions can be run.

This approach is known as allow-listing and is considered a security best practice for enterprises using GitHub Actions.

Principle of Least Privilege: By creating a list of allowed actions, the enterprise operates on the principle of least privilege. Any new, unvetted, or potentially malicious action from the marketplace is automatically blocked until it is explicitly reviewed and added to the allow-list.

Security and Control: This method provides the highest level of control over the code running inside the enterprise environment. The enterprise maintains a curated set of actions known to be safe, audited, and compliant with internal standards.

Why Other Options Are incorrect:

A. Create a list of the actions that are restricted... Every other action can be run (Deny-listing): This is called

deny-listing or block-listing. It's less secure because it requires constant maintenance to keep up with every new or risky action that is published. If a bad actor publishes a new action, it can be run until the enterprise catches and adds it to the restricted list.

B. It is not possible...: This is incorrect. GitHub Enterprise offers granular controls to manage the usage of marketplace and external actions.

C. Create a list that is maintained as a .yml file...: While you could maintain a list this way, the enterprise policy settings (as mentioned in D) are the dedicated and centralized method provided by GitHub Enterprise for enforcing this rule across all organizations and repositories. Using the policy mechanism is authoritative and cannot be easily bypassed by repository settings.

Question: 85 **Deep Dumps**

When creating and managing custom actions in an enterprise setting, which of the following is considered a best practice?

A. creating a separate repository for each action so that the version can be managed independently
B. creating a separate branch in application repositories that only contains the actions
C. creating a single repository for all custom actions so that the versions for each action are all the same
D. including custom actions that other teams need to reference in the same repository as application code

Answer: C

Explanation:

Justification for Answer Choice C: Creating a single repository for all custom actions so that the versions for each action are all the same

In an enterprise setting, managing custom GitHub Actions effectively is crucial for maintainability, security, and reusability. Creating a single repository dedicated solely to custom actions is generally considered a best practice for several reasons:

1. **Centralized Management:** A single repository provides a central location for all custom actions, making them easier to discover, manage, and update. Teams can easily browse and find the actions they need.
2. **Version Control:** While the original answer suggests keeping all versions of actions the same, this is actually incorrect. A single repository doesn't enforce that versions must be the same; good practice is to version each action independently via tags or branches and the single repository allows for management of those different versions. Each action within the single repository should use semantic versioning (e.g., v1, v2, v1.0.0) to allow consumers to lock in on a version, ensuring that changes don't break their workflows unexpectedly.
3. **Reusability:** Housing all custom actions in one place encourages reusability across different teams and projects. This reduces code duplication and promotes consistency in workflows.
4. **Security:** A dedicated repository simplifies security audits and access control. You can manage permissions for the actions repository separately from application repositories, ensuring that only authorized personnel can modify or deploy actions.
5. **Collaboration:** A single repository fosters collaboration among developers working on actions. Teams can contribute to and improve existing actions, leading to a more robust and well-maintained set of tools.
6. **Dependency Management:** The action repository can also host any shared dependencies needed by the actions, which simplifies dependency management across the enterprise.

Why the other options are less ideal:

A. creating a separate repository for each action: While this provides excellent isolation and versioning control, it can quickly become unwieldy in a large enterprise with many custom actions. Discoverability and centralized management are compromised. The overhead of maintaining numerous repositories outweighs the benefits.
B. creating a separate branch in application repositories: This tightly couples actions to specific applications, hindering reusability and making it difficult to manage actions across the organization. It also violates the principle of separation of concerns.
D. including custom actions that other teams need to reference in the same repository as application code: This suffers from the same problems as option B, leading to tight coupling, lack of reusability, and increased complexity in application repositories.

Supporting Documentation:

While there isn't a single document explicitly stating "use a single repository," the general principles of code reuse, dependency management, and organizational best practices support this approach. The GitHub Actions documentation provides guidance on creating and using custom actions. It is through understanding these concepts that the benefit of a single repository is realized.

Creating a composite action:https://docs.github.com/en/actions/creating-actions/creating-a-composite-action
Metadata syntax for GitHub Actions:https://docs.github.com/en/actions/creating-actions/metadata-syntax-for-github-actions

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers

Future is Secured
100% Pass Guarantee

24/7 Customer Support
Mail us - support@deepdumps.com

Verified Updates
Lifetime Updates!

Total: **87 Questions**
Link: https://deepdumps.com/papers/microsoft/gh-200